

RAPPORT DE STAGE

Cycle Ingénieur — Filière Informatique et Réseaux — 3ème Année



Conception et mise en place d'une solution centralisée de supervision Zabbix

Stage réalisé au sein de MedTech Group

Réalisé par :

MOUKRIM Wissal

Encadré par :

M. Rachid Rebik — Encadrant
professionnel, MedTech Group

M. Anas Mancour-Billah — Encadrant
académique, EMSI

Année universitaire 2025 — 2026

Dédicace

Je dédie ce modeste travail, tout d'abord à Allah, le Tout-Puissant, à qui je dois tout, et qui m'a donné la force et la persévérance nécessaires pour mener à bien ce travail.

Je le dédie ensuite à toutes les personnes qui m'ont accompagnée et soutenue tout au long de mon parcours académique et durant la réalisation de ce stage.

À mes parents, pour leur soutien constant, leurs encouragements et les sacrifices consentis afin que je puisse mener à bien mes études d'ingénieur dans les meilleures conditions.

À ma famille et à mes proches, pour leur confiance et leur patience tout au long de ce parcours.

À mes enseignants de l'EMSI, pour la qualité de la formation dispensée et les compétences transmises, qui constituent aujourd'hui le socle de ma pratique d'ingénieure en devenir.

À toute personne ayant contribué, de près ou de loin, à la réussite de ce stage et à l'élaboration de ce rapport.

Remerciements

Avant toute chose, je remercie Allah, le Tout-Puissant, pour m'avoir accordé la santé, la volonté, la patience et la capacité de mener à bien ce stage et de réaliser ce rapport dans de bonnes conditions.

Au terme de ce stage d'observation, je tiens à exprimer ma sincère reconnaissance à l'ensemble des personnes qui ont contribué, directement ou indirectement, à son bon déroulement.

Mes remerciements s'adressent en premier lieu à M. Rachid Rebik, mon encadrant professionnel au sein de MedTech Group, pour la confiance qu'il m'a accordée, la qualité de son encadrement technique et sa disponibilité tout au long de cette expérience. Ses explications sur l'administration Linux, la virtualisation, la conteneurisation et la supervision centralisée ont constitué un apport déterminant dans ma compréhension du sujet et dans la réalisation du laboratoire présenté dans ce rapport.

Je remercie également l'ensemble de l'équipe Infrastructure, Systèmes et DevOps de MedTech Group, pour son accueil et le climat de travail favorable qui a permis à ce stage de se dérouler dans de bonnes conditions d'apprentissage.

Mes remerciements vont aussi à l'ensemble du corps professoral et administratif de mon école (EMSI), pour la formation théorique et pratique dispensée tout au long de mon cursus, qui m'a permis d'aborder ce stage avec les prérequis nécessaires en systèmes, réseaux et virtualisation.

Enfin, je remercie ma famille et mes proches pour leur soutien indéfectible, ainsi que toute personne ayant contribué de près ou de loin à l'aboutissement de ce travail.

Résumé

Ce rapport rend compte d'un stage d'observation effectué au sein de MedTech Group, groupe marocain de conseil, d'ingénierie et de services dans les technologies de l'information, dans le cadre de la formation d'ingénieur en Informatique et Réseaux dispensée par l'EMSI. Le stage s'inscrit dans le périmètre Infrastructure, Systèmes et DevOps du groupe et porte sur la conception et la mise en place d'une solution centralisée de supervision des infrastructures systèmes et réseaux, à l'aide de l'outil open source Zabbix.

Face à l'importance croissante de la disponibilité des services informatiques et à la nécessité d'une détection proactive des incidents, ce travail propose la conception d'un laboratoire de supervision reproduisant, à l'échelle d'un environnement virtualisé contrôlé, les principes appliqués dans une infrastructure professionnelle. L'environnement repose sur deux machines virtuelles Ubuntu Server déployées sous VMware : la première héberge, via Docker et Docker Compose, la pile applicative Zabbix Server, MySQL et Zabbix Web ; la seconde, équipée de Zabbix Agent 2, constitue l'hôte supervisé.

La démarche suivie couvre l'étude de l'existant et le choix argumenté de Zabbix parmi plusieurs solutions de supervision, la conception de l'architecture logique, physique et réseau de la solution, sa mise en œuvre pas à pas — de l'installation des machines virtuelles jusqu'à la configuration des hôtes, des modèles (templates), des indicateurs (items), des seuils d'alerte (triggers) et du tableau de bord — ainsi que l'intégration de deux canaux de notification, l'un par messagerie électronique (Gmail) et l'autre par webhook vers Microsoft Teams via Power Automate. Une phase de tests permet de valider la chaîne complète de supervision, depuis la collecte des métriques jusqu'à la réception effective des alertes.

Ce travail a permis de développer des compétences concrètes en administration Linux, en virtualisation, en conteneurisation et en supervision informatique, tout en mettant en évidence les limites du laboratoire réalisé et les perspectives d'évolution nécessaires à un déploiement en environnement de production.

Mots-clés : supervision informatique, Zabbix, Docker, Docker Compose, virtualisation, VMware, Ubuntu Server, alerting, Microsoft Teams, Gmail.

Abstract

This report presents an observation internship carried out within MedTech Group, a Moroccan consulting, engineering and IT services group, as part of the engineering program in Computer Science and Networks at EMSI. The internship was attached to the Infrastructure, Systems and DevOps scope of the group and focused on the design and implementation of a centralized monitoring solution for system and network infrastructures, based on the open-source tool Zabbix.

Given the growing importance of IT service availability and the need for proactive incident detection, this work presents the design of a monitoring laboratory reproducing, within a controlled virtualized environment, the principles applied in a professional infrastructure. The environment relies on two Ubuntu Server virtual machines deployed under VMware: the first hosts, through Docker and Docker Compose, the Zabbix Server, MySQL and Zabbix Web application stack, while the second, equipped with Zabbix Agent 2, acts as the monitored host.

The approach followed covers the study of existing monitoring solutions and the justified choice of Zabbix, the design of the logical, physical and network architecture of the solution, its step-by-step implementation — from virtual machine installation to host, template, item, trigger and dashboard configuration — and the integration of two notification channels, one by email (Gmail) and the other by webhook to Microsoft Teams through Power Automate. A test phase validates the complete monitoring chain, from metric collection to the effective reception of alerts.

This work made it possible to develop concrete skills in Linux administration, virtualization, containerization and IT monitoring, while highlighting the limitations of the laboratory implemented and the evolutions required for a deployment in a production environment.

Keywords: IT monitoring, Zabbix, Docker, Docker Compose, virtualization, VMware, Ubuntu Server, alerting, Microsoft Teams, Gmail.

Liste des acronymes

Acronyme	Signification
API	Application Programming Interface
CPU	Central Processing Unit
DHCP	Dynamic Host Configuration Protocol
DNS	Domain Name System
EMSI	École Marocaine des Sciences de l'Ingénieur
HA	Haute Disponibilité
HTTP / HTTPS	HyperText Transfer Protocol (Secure)
IP	Internet Protocol
JSON	JavaScript Object Notation
LAN	Local Area Network
NAT	Network Address Translation
OS	Operating System (Système d'exploitation)
RAM	Random Access Memory
SLA	Service Level Agreement
SMTP	Simple Mail Transfer Protocol
SNMP	Simple Network Management Protocol
SQL	Structured Query Language
SSH	Secure Shell
TCP/IP	Transmission Control Protocol / Internet Protocol
TLS	Transport Layer Security
VM	Virtual Machine (Machine Virtuelle)
VMware	Éditeur et suite logicielle de virtualisation (VMware Workstation)

Liste des figures

- Figure 1 — Repères historiques de MedTech Group
- Figure 2 — Organisation et panorama des filiales de MedTech Group
- Figure 3 — Implantation de MedTech Group
- Figure 4 — Comparaison des principales solutions de supervision
- Figure 5 — Architecture générale du projet
- Figure 6 — Architecture logique de la solution
- Figure 7 — Architecture physique du laboratoire
- Figure 8 — Architecture réseau du laboratoire
- Figure 9 — Flux de notification par Gmail
- Figure 10 — Flux de notification par Microsoft Teams
- Figure 11 — Chaîne de traitement d'une alerte Zabbix
- Figure 12 — Vérification de l'environnement VMware et des machines virtuelles
- Figure 13 — Configuration réseau et test de connectivité (ping)
- Figure 14 — Vérification de l'installation de Docker
- Figure 15 — Conteneurs actifs de la pile Zabbix
- Figure 16 — Interface web Zabbix — page d'authentification
- Figure 17 — Message d'indisponibilité de l'agent Zabbix
- Figure 18 — Hôte Ubuntu-Client VM après résolution de l'incident
- Figure 19 — Fiche de l'hôte Ubuntu-Client dans Zabbix
- Figure 20 — Modèle (template) associé à l'hôte supervisé
- Figure 21 — Tableau de bord (Dashboard) Zabbix
- Figure 22 — Configuration du Media Type Email (Gmail)
- Figure 23 — Réception d'une notification par courrier électronique
- Figure 24 — Configuration du Media Type Webhook Microsoft Teams
- Figure 25 — Réception d'une notification sur Microsoft Teams

Liste des tableaux

Tableau 1 — Composants de la solution et rôles associés

Tableau 2 — Plan de tests fonctionnels et techniques

Tableau 3 — Validation des objectifs du stage

Tableau 4 — Synthèse des difficultés rencontrées et solutions apportées

Tableau 5 — Tests complémentaires réalisés en environnement de laboratoire

Table des matières

Dédicace	2
Remerciements	3
Résumé	4
Abstract.....	5
Liste des acronymes	6
Liste des figures.....	7
Liste des tableaux	8
Table des matières	9
Introduction générale.....	12
Chapitre 1 — Présentation de l’organisme d’accueil	14
1.1. Présentation générale de MedTech Group	14
1.2. Repères historiques.....	14
1.3. Domaines d’intervention et filiales	15
1.4. Chiffres clés et implantation.....	16
1.5. Mission, vision et valeurs	17
1.6. Département d’accueil et contexte du stage	17
Conclusion du chapitre	18
Chapitre 2 — Étude préalable et contexte technique	19
2.1. Étude de l’existant	19
2.2. Problématique.....	19
2.3. Objectifs du stage	20
2.4. Besoins fonctionnels.....	20
2.5. Besoins non fonctionnels.....	21
2.6. Cahier des charges synthétique	21
2.7. Généralités sur la supervision informatique	21
2.8. Étude comparative des solutions de supervision	21
2.9. Choix de Zabbix — justification	22
2.10. Technologies support retenues	23
2.10.1. VMware	23
2.10.2. Ubuntu Server.....	23
2.10.3. Docker	23
2.10.4. Docker Compose	23
Conclusion du chapitre	23

Chapitre 3 — Conception de la solution	24
3.1. Architecture générale.....	24
3.2. Architecture logique	25
3.3. Architecture physique du laboratoire	25
3.4. Architecture réseau	26
3.5. Composants de la solution.....	27
3.6. Rôle du Zabbix Server.....	28
3.7. Rôle de Zabbix Agent 2.....	28
3.8. Rôle de MySQL.....	28
3.9. Rôle de Zabbix Web.....	29
3.10. Docker	29
3.11. Docker Compose	29
3.12. Collecte des données	29
3.13. Items	29
3.14. Triggers.....	30
3.15. Problems	30
3.16. Actions.....	30
3.17. Notifications	30
3.18. Intégration Gmail.....	30
3.19. Intégration Microsoft Teams Workflow.....	31
3.20. Sécurité	32
3.21. Flux de communication global	32
Conclusion du chapitre	33
Chapitre 4 — Mise en œuvre.....	34
4.1. Préparation de l’environnement de virtualisation.....	34
4.2. Installation et validation de Docker.....	35
4.3. Déploiement de la pile Zabbix via Docker Compose.....	36
4.4. Intégration de la machine cliente supervisée.....	37
4.4.1. Installation et configuration de l’agent Zabbix	37
4.4.2. Ajout de l’hôte et association du modèle	37
4.4.3. Difficulté rencontrée : indisponibilité de l’agent Zabbix	37
4.5. Configuration du tableau de bord et des alertes	40
4.6. Intégration des canaux de notification.....	40
4.6.1. Notification par Gmail.....	40
4.6.2. Notification par Microsoft Teams	42

4.7. Synthèse des étapes de mise en œuvre	43
Conclusion du chapitre	43
Chapitre 5 — Tests, résultats et validation.....	44
5.1. Stratégie de test.....	44
5.2. Plan de tests	44
5.3. Validation des objectifs	45
5.4. Analyse des résultats	46
5.5. Difficultés et analyse critique	46
5.6. Limites	46
Conclusion du chapitre	46
Chapitre 6 — Bilan et perspectives	47
6.1. Compétences techniques acquises	47
6.2. Compétences professionnelles.....	47
6.3. Difficultés rencontrées et solutions apportées	47
6.4. Apports du stage	48
6.5. Limites du travail réalisé	48
6.6. Perspectives d'évolution.....	48
Conclusion du chapitre	49
Conclusion générale	50
Bibliographie	51
Articles scientifiques	51
Documentation technique	51
Annexes	52
Annexe A — Commandes Linux utiles.....	52
Annexe B — Commandes Docker	52
Annexe C — Extrait du fichier Docker Compose.....	53
Annexe D — Tableau complémentaire de tests	54

Introduction générale

La transformation numérique s'impose aujourd'hui comme un axe stratégique majeur pour les organisations. Les systèmes d'information, autrefois de simples outils de support, portent désormais les applications métiers, hébergent les données critiques et conditionnent la continuité des activités. La disponibilité, la performance et la fiabilité des infrastructures deviennent ainsi des enjeux déterminants, au même titre que la sécurité des données.

Cette dépendance s'accompagne d'une complexité technique croissante : multiplication des serveurs, des services applicatifs, des équipements réseau et des environnements virtualisés ou conteneurisés, rendant difficile le maintien d'une visibilité complète sur l'état de l'infrastructure. Une panne non détectée ou une ressource saturée peut engendrer des interruptions coûteuses, tant sur le plan opérationnel que sur l'image de l'organisation. Il devient donc indispensable de dépasser une gestion réactive des incidents au profit d'une démarche proactive, fondée sur l'observation continue et la génération d'alertes précoces.

C'est dans cette problématique de supervision informatique que s'inscrit ce rapport, qui rend compte d'un stage d'observation effectué au sein de MedTech Group, groupe marocain reconnu dans le conseil, l'ingénierie et les services liés aux technologies de l'information. Rattaché au périmètre Infrastructure, Systèmes et DevOps du groupe, le stage a porté sur la conception d'une solution centralisée de supervision fondée sur Zabbix, afin d'illustrer comment une telle solution améliore la disponibilité des services et favorise une détection proactive des incidents.

Les travaux présentés ont été réalisés au sein d'un environnement de laboratoire virtualisé, constitué de deux machines virtuelles sous VMware. Il ne s'agit donc pas d'une intervention sur l'infrastructure de production de MedTech Group, mais de la conception d'un environnement contrôlé reproduisant, à échelle réduite, les mécanismes d'une infrastructure professionnelle de supervision — une approche cohérente avec la nature de ce stage d'observation.

L'objectif de ce rapport est double : démontrer une véritable démarche d'ingénieur, structurée autour de la compréhension, de l'analyse, de la conception, du déploiement, du test et de l'amélioration continue ; et transmettre au lecteur une compréhension claire du contexte,

des choix techniques, de l'architecture, des difficultés rencontrées et des résultats obtenus, sans jamais confondre réalisations effectives et perspectives d'évolution.

La méthodologie suivie a consisté à progresser par étapes successives, chaque brique technique étant validée avant la suivante : préparation de l'environnement de virtualisation, installation de Docker, déploiement de la pile Zabbix, intégration de l'hôte supervisé, configuration des alertes, puis mise en place des notifications. Cette progression a permis d'isoler et de résoudre les difficultés rencontrées, notamment celle relative à l'agent Zabbix, avant d'aboutir à une chaîne de supervision fonctionnelle et testée.

Ce rapport est organisé en six chapitres. Le premier présente MedTech Group et le contexte du stage. Le deuxième expose l'étude préalable : existant, problématique, objectifs, besoins et comparaison des solutions de supervision justifiant le choix de Zabbix. Le troisième détaille la conception de la solution retenue. Le quatrième relate chronologiquement sa mise en œuvre. Le cinquième présente les tests et leur analyse critique. Le sixième dresse le bilan du stage et les perspectives d'évolution.

Chapitre 1 — Présentation de l'organisme d'accueil

Ce premier chapitre présente l'organisme au sein duquel s'est déroulé le stage, MedTech Group, à travers son historique, ses domaines d'intervention, ses valeurs et son organisation. Il précise ensuite le contexte concret dans lequel s'est inscrit le stage d'observation, avant d'introduire la problématique technique qui sera développée dans les chapitres suivants.

1.1. Présentation générale de MedTech Group

MedTech Group est un groupe marocain positionné parmi les acteurs de référence du conseil, de l'ingénierie et des services dans les technologies de l'information. Il revendique une présence significative dans les grands projets informatiques de l'administration marocaine, ainsi qu'une implantation durable au sein des principales institutions financières du pays.

Le groupe s'appuie sur plus de trente années de développement continu dans l'industrie des technologies de l'information au Maroc, et poursuit une stratégie d'expansion internationale, avec une présence commerciale en Europe où plusieurs grands comptes exploitent ses plateformes technologiques, notamment dans le Big Data et l'intelligence artificielle.

1.2. Repères historiques

L'histoire du groupe débute en 1989 avec la création d'Omnidata, premier revendeur à valeur ajoutée Oracle au Maroc, qui s'impose dès 1994 comme premier intégrateur ERP du pays, puis pionnier de l'architecture 3-tiers en 1997 et des solutions de Business Intelligence en 1999. Le groupe se développe ensuite par acquisitions structurantes : NCRM (2000), AeBDM (2003), Forum (2010), Aaxis Group en France et au Benelux (2015). En 2016, l'ouverture du capital à Mediterrania Capital Partners en fait l'actionnaire de référence de S2M et de Dimension Data Maroc, avant les acquisitions de Credon et d'OppScience en 2017.



Source : MedTech Group — Matériaux institutionnels et informations publiques.

Figure 1 — Repères historiques de MedTech Group

Cette trajectoire illustre une croissance fondée sur l'intégration progressive de compétences complémentaires — infrastructures, applications métiers, paiement, innovation — qui structure encore aujourd'hui l'organisation du groupe.

1.3. Domaines d'intervention et filiales

L'offre du groupe s'articule autour de quatre grands domaines d'intervention complémentaires, portés par un ensemble de filiales spécialisées.

- Les solutions IT, regroupant le développement spécifique, l'édition de logiciels et l'intégration de solutions métiers, portées notamment par les filiales Omndata, Omndata Business Solutions, OmniAcademy, Omnishore, Dial Technologies, Aaxis, Credon, OppScience et SGEvT.
- Les solutions d'infrastructures et de courant faible, couvrant le matériel, les logiciels et les solutions associées (Cloud, sécurité, télécom, courant faible), autour des filiales Forum, DDM et AeBDM.
- Les solutions de paiement, regroupant équipements, services et édition logicielle autour des filiales NCRM et S2M, cette dernière étant une société monétique cotée à la Bourse de Casablanca.
- L'innovation transversale, portée par les technologies de cybersécurité, de Blockchain, de Big Data et d'intelligence artificielle, qui irriguent l'ensemble des autres domaines d'activité du groupe.

C'est au sein de ce premier domaine, et plus particulièrement des activités d'intégration d'infrastructures et de supervision portées par Omnidata, que s'inscrit le stage présenté dans ce rapport.

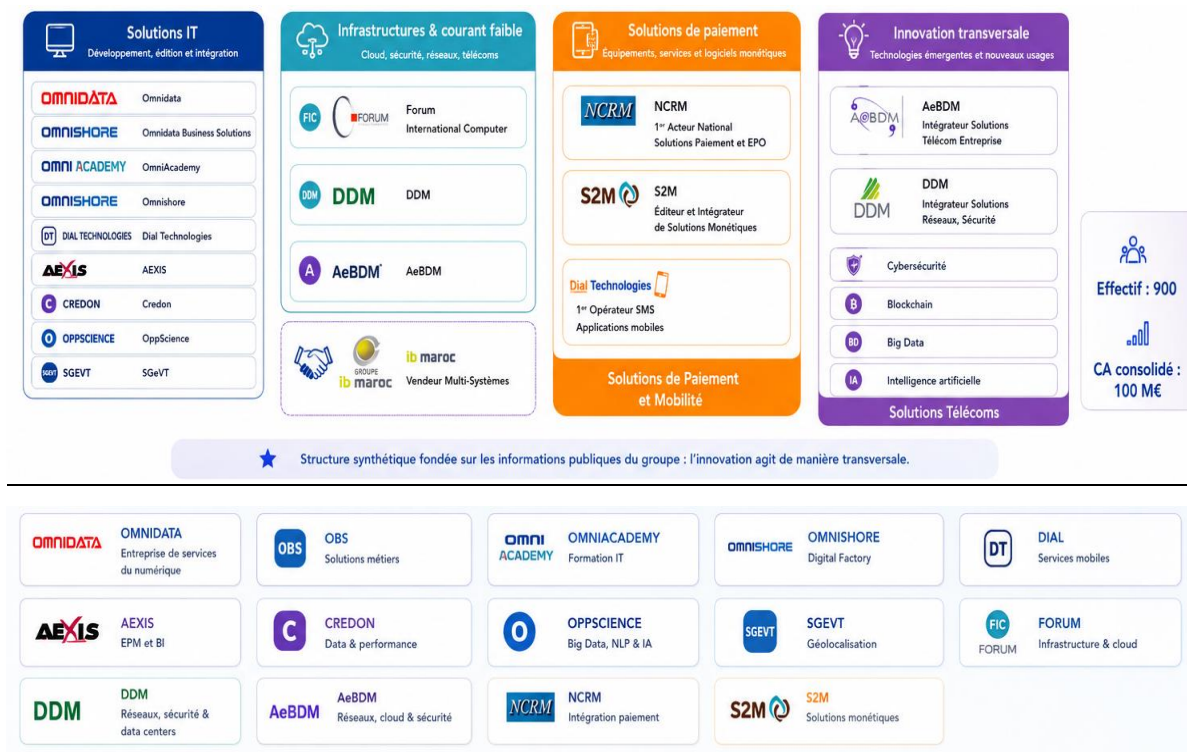


Figure 2 — Organisation et panorama des filiales de MedTech Group

1.4. Chiffres clés et implantation

MedTech Group compte plusieurs centaines de collaborateurs et réalise un chiffre d'affaires de plusieurs dizaines de millions d'euros, au Maroc et à l'international. Le groupe dispose d'un large réseau de clients et de partenaires, renforcé par la présence internationale de sa filiale S2M dans la monétique.



Figure 3 — Implantation

1.5. Mission, vision et valeurs

MedTech Group présente sa mission comme celle d'accompagner ses clients dans l'assimilation de l'évolution technologique, en leur fournissant des infrastructures numériques avancées, au service de l'économie marocaine et de la démocratisation de l'accès aux technologies de l'information. Sa vision consiste à accompagner la transformation digitale de ses clients dans un contexte où le numérique occupe une place croissante.

Le groupe met en avant trois valeurs structurantes : l'innovation, au cœur du développement de ses solutions ; l'esprit de collaboration, en interne comme avec ses filiales et partenaires ; et la transparence, associée à l'intégrité, comme principes fondamentaux de sa gouvernance.

1.6. Département d'accueil et contexte du stage

Au regard du sujet confié, le stage a été rattaché fonctionnellement au périmètre Infrastructure, Systèmes et DevOps de MedTech Group.

Ce périmètre couvre les activités d'intégration d'infrastructures logicielles et matérielles, de virtualisation, de plateformes open source et de supervision développées au sein du groupe, notamment par Omnidata. Ses missions consistent à préparer et administrer les environnements techniques, automatiser les déploiements, surveiller la disponibilité des systèmes et contribuer à la détection des incidents.

L'équipe encadrante s'est composée principalement de M. Rachid Rebik, encadrant professionnel et interlocuteur technique du stage, qui a défini les orientations du sujet, mis à disposition les ressources nécessaires et validé les choix techniques tout au long des travaux, portant notamment sur l'administration Linux, la virtualisation, la conteneurisation et la supervision centralisée.

Dans ce cadre, j'ai été accueillie pour la durée de mon stage d'observation, sous l'encadrement de M. Rachid Rebik, dans une logique de découverte des pratiques de supervision informatique, avec pour finalité la conception d'un laboratoire reproduisant, à l'échelle d'un environnement contrôlé, les principes d'une infrastructure de production.

Ce stage relève d'un stage d'observation : les travaux décrits ont été réalisés dans un environnement de laboratoire constitué de deux machines virtuelles, et ne correspondent pas à une intervention directe sur l'infrastructure de production de l'entreprise.

Conclusion du chapitre

La présentation de MedTech Group met en évidence un groupe structuré autour de filiales complémentaires et doté d'une expérience étendue dans l'intégration de solutions informatiques. Le chapitre suivant précise le contexte technique du projet confié, à partir d'une étude de l'existant justifiant la nécessité d'une solution de supervision centralisée.

Chapitre 2 — Étude préalable et contexte technique

Ce chapitre présente la démarche d'analyse préalable à la conception de la solution : étude de l'existant, problématique, besoins fonctionnels et non fonctionnels, puis justification comparative du choix de Zabbix.

2.1. Étude de l'existant

Dans de nombreuses organisations, la surveillance des systèmes repose encore sur des pratiques manuelles ou fragmentées : vérifications ponctuelles, journaux dispersés, ou remontée d'incidents par les utilisateurs eux-mêmes. Cette approche réactive montre vite ses limites : l'absence de vision centralisée retarde la détection des anomalies et allonge le délai de résolution des incidents.

Cette problématique concerne aussi bien les environnements physiques que virtualisés et conteneurisés, de plus en plus répandus. La multiplication des machines virtuelles et des conteneurs accroît le nombre d'éléments à surveiller simultanément, ce qui justifie la mise en place d'une solution de supervision centralisée capable de collecter et d'analyser en continu les indicateurs de l'infrastructure.

2.2. Problématique

La problématique de ce projet peut être formulée ainsi : comment concevoir une solution de supervision centralisée, capable de collecter en temps réel les indicateurs de disponibilité et de performance, et de déclencher automatiquement des alertes pertinentes, afin de permettre une détection proactive des incidents plutôt qu'une gestion réactive ?

Cette problématique s'articule autour de trois dimensions : technique (choix et configuration des outils de collecte), organisationnelle (transmission des alertes sur les canaux réellement utilisés) et méthodologique (passer d'un besoin exprimé à une solution testée).

2.3. Objectifs du stage

Le stage poursuit un objectif général : concevoir et mettre en place, au sein d'un laboratoire virtualisé, une solution centralisée de supervision fondée sur Zabbix, illustrant les mécanismes améliorant la disponibilité des services et la détection proactive des incidents.

- Comprendre les principes généraux de la supervision informatique et les distinguer des notions voisines de monitoring et d'alerting.
- Étudier et comparer les principales solutions de supervision disponibles, afin de justifier techniquement le choix de Zabbix.
- Concevoir une architecture de supervision cohérente, reposant sur la virtualisation, la conteneurisation et une pile applicative Zabbix conteneurisée.
- Déployer et configurer cette architecture au sein d'un environnement de laboratoire constitué de deux machines virtuelles.
- Intégrer un hôte supervisé à l'aide de Zabbix Agent 2 et configurer la collecte des métriques associées.
- Mettre en place des mécanismes d'alerte automatisés, notifiés par courrier électronique et par Microsoft Teams.
- Tester et valider l'ensemble de la chaîne de supervision, de la collecte des données jusqu'à la réception des notifications.

2.4. Besoins fonctionnels

Sur le plan fonctionnel, la solution doit collecter automatiquement des indicateurs techniques (disponibilité, charge processeur, mémoire, disque, état des services) à intervalle régulier, définir des modèles réutilisables (templates) et des seuils d'alerte (triggers) déclenchant un état de problème.

Elle doit offrir une visualisation centralisée sous forme de tableau de bord, et permettre l'envoi automatique de notifications via au moins deux canaux distincts : la messagerie électronique et Microsoft Teams.

2.5. Besoins non fonctionnels

Sur le plan non fonctionnel, la solution doit reposer sur des composants open source, s'appuyer sur une architecture modulaire et reproductible, et rester simple à administrer, compatible avec le cadre pédagogique d'un stage d'observation tout en respectant les principes d'un contexte professionnel réel.

2.6. Cahier des charges synthétique

En synthèse, le cahier des charges consiste à mettre en place, sous VMware, une solution de supervision fondée sur Zabbix, déployée via Docker et Docker Compose, capable de superviser un hôte distant via Zabbix Agent 2, de restituer les métriques sous forme de tableau de bord, de déclencher des alertes automatiques et de les notifier par courrier électronique et par Microsoft Teams.

2.7. Généralités sur la supervision informatique

La supervision informatique, ou monitoring, désigne l'ensemble des techniques permettant d'observer en continu l'état d'un système d'information. Elle repose sur la collecte régulière de métriques telles que la disponibilité, la charge processeur, l'occupation mémoire, l'espace disque ou le temps de réponse.

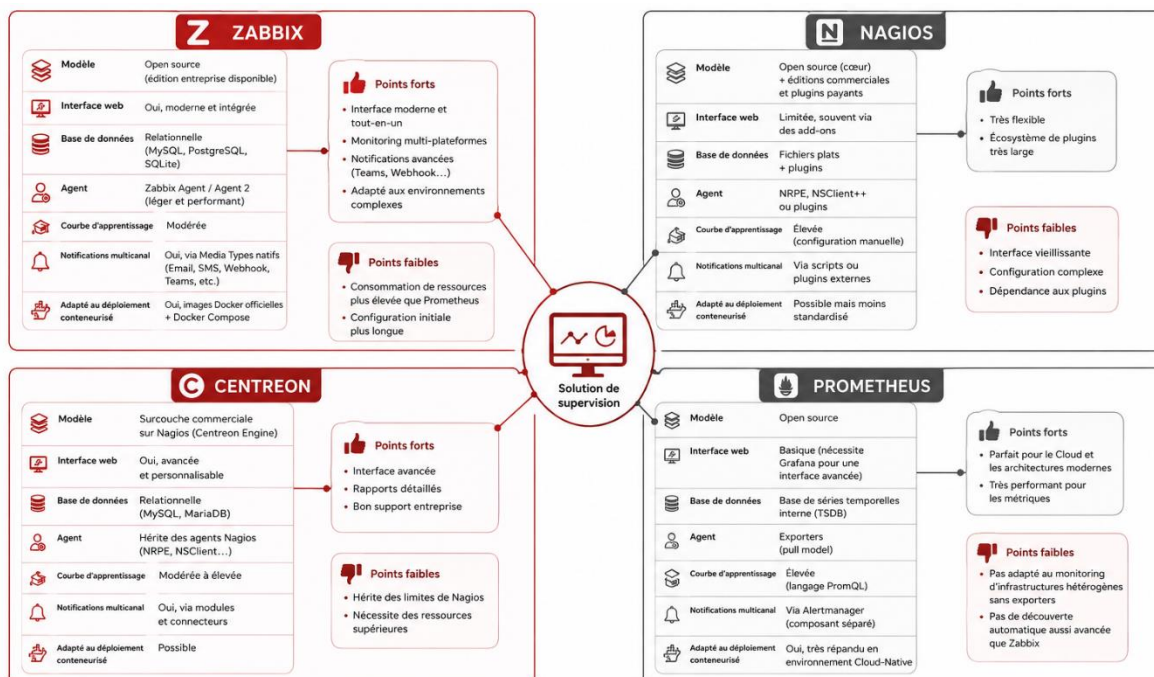
L'alerting prolonge le monitoring en définissant des règles, généralement fondées sur des seuils, qui transforment une observation technique en signal exploitable. La pertinence d'un système de supervision se mesure donc à sa capacité à transformer les données en alertes utiles, transmises aux bonnes personnes au bon moment.

On distingue la supervision réactive, où l'incident est constaté après coup, de la supervision proactive, où les signaux avant-coureurs d'une dégradation sont détectés avant que l'incident n'affecte les utilisateurs. C'est cette seconde approche que vise à illustrer ce projet.

2.8. Étude comparative des solutions de supervision

Plusieurs solutions de supervision sont aujourd'hui largement utilisées en entreprise. Le tableau suivant compare quatre solutions représentatives — Zabbix, Nagios, Centreon et Prometheus — sur des critères pertinents pour ce projet.

Figure 4 — Comparaison des principales solutions de supervision



Nagios, historiquement fondateur du domaine, repose sur une configuration textuelle et une interface limitée. Centreon, construit sur le moteur Nagios, apporte une interface plus moderne mais dépend d'un écosystème plus lourd. Prometheus, répandu en environnement Cloud-natif, adopte un modèle de collecte par interrogation et nécessite Grafana pour la visualisation et Alertmanager pour les notifications, ce qui multiplie les composants à administrer.

2.9. Choix de Zabbix — justification

Zabbix a été retenu pour plusieurs raisons. Il propose nativement l'ensemble des briques d'une solution complète — collecte, interface web, base de données, moteur d'alerte et notifications multicanal — sans assembler plusieurs outils distincts, contrairement à l'écosystème Prometheus/Grafana/Alertmanager. Ce choix correspond bien au cadre d'un stage d'observation, où l'objectif est de maîtriser une chaîne complète plutôt que de multiplier les composants.

Zabbix dispose d'images Docker officielles pour tous ses composants, facilitant son déploiement conteneurisé via Docker Compose. Son mécanisme de Media Types flexible permet de configurer aussi bien l'email que des webhooks vers des outils tiers comme

Microsoft Teams. Sa documentation exhaustive et sa communauté active en font un choix pédagogiquement adapté.

2.10. Technologies support retenues

2.10.1. VMware

VMware Workstation a été retenu pour la virtualisation du laboratoire. Il permet de créer, sur une machine unique, plusieurs machines virtuelles isolées disposant chacune de leurs ressources propres, tout en simulant un environnement réseau réaliste — reproduisant ainsi la logique d’une infrastructure multi-serveurs sans matériel dédié.

2.10.2. Ubuntu Server

Ubuntu Server a été choisi comme système d’exploitation des machines virtuelles, pour sa large adoption professionnelle, sa documentation étendue et sa bonne compatibilité avec Docker. Sa nature open source en fait un choix cohérent avec la philosophie du projet.

2.10.3. Docker

Docker est une plateforme de conteneurisation permettant d’exécuter des applications dans des environnements isolés et légers, partageant le noyau du système hôte. Dans ce projet, Docker isole chaque composant de la pile Zabbix dans un environnement reproductible et indépendant de la machine hôte.

2.10.4. Docker Compose

Docker Compose décrit, dans un fichier unique, l’ensemble des conteneurs interdépendants et leurs paramètres (images, variables d’environnement, volumes, réseaux, dépendances de démarrage). Il permet de déployer toute la pile Zabbix en une seule commande, garantissant la cohérence et la reproductibilité de l’environnement.

Conclusion du chapitre

Cette étude préalable a permis de formuler la problématique, de dégager les besoins et de justifier le choix de Zabbix ainsi que des technologies support retenues. Le chapitre suivant s’appuie sur ces choix pour détailler la conception technique de la solution.

Chapitre 3 — Conception de la solution

Ce chapitre présente la conception technique de la solution, à travers ses architectures générale, logique, physique et réseau, puis détaille le rôle de chacun des composants et les flux d'information, en particulier les deux chaînes de notification.

3.1. Architecture générale

L'architecture générale repose sur une séparation claire entre l'environnement de virtualisation, qui héberge les machines virtuelles, et la pile applicative de supervision, déployée sous forme de conteneurs. Le schéma ci-dessous résume l'enchaînement des couches technologiques, de l'hyperviseur jusqu'à l'interface web.

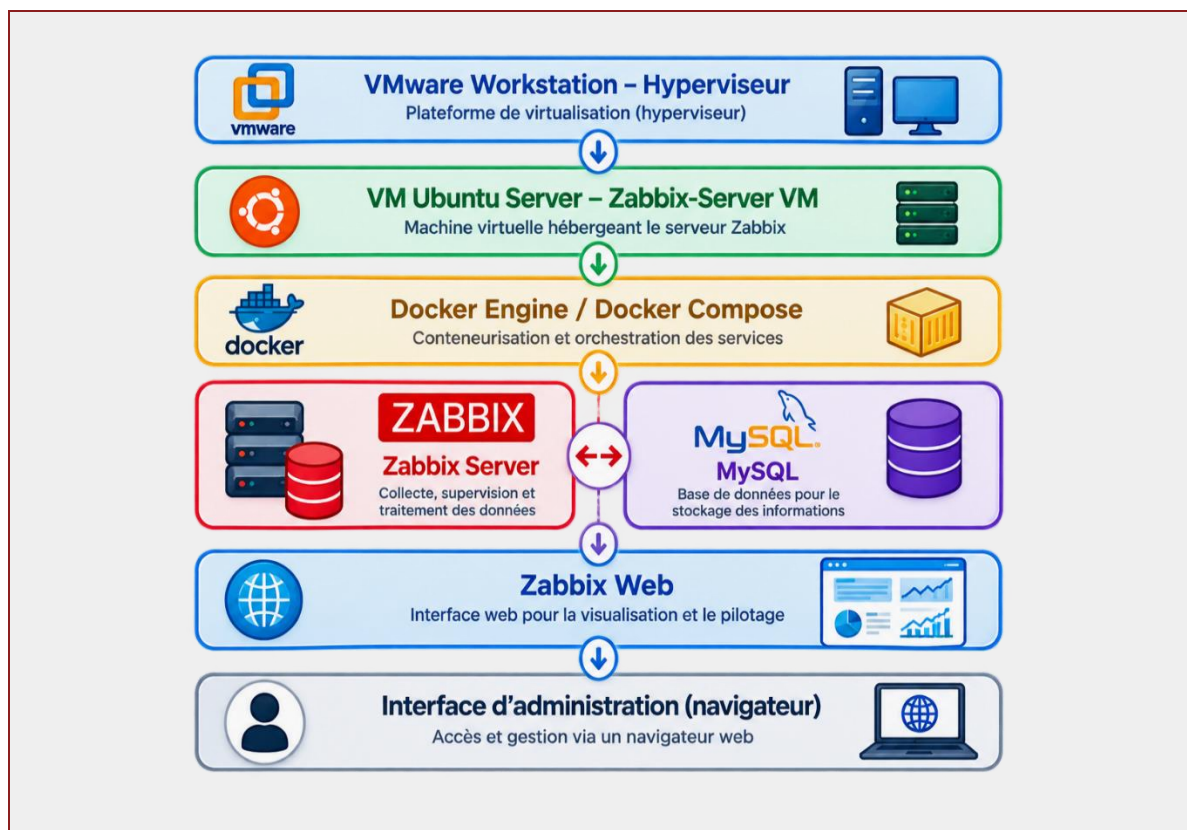


Figure 5 — Architecture générale du projet

Cette organisation en couches illustre la logique retenue : l'hyperviseur isole l'environnement multi-serveurs ; Ubuntu Server constitue le socle de la conteneurisation ; Docker Compose orchestre le démarrage des trois conteneurs applicatifs ; l'interface web expose enfin les fonctionnalités de configuration et de visualisation.

3.2. Architecture logique

L'architecture logique met en évidence les relations fonctionnelles entre composants. Deux flux structurent cette architecture : le flux de collecte, par lequel l'agent transmet ses métriques au serveur Zabbix ; et le flux de restitution, par lequel le serveur expose les données et déclenche les alertes.

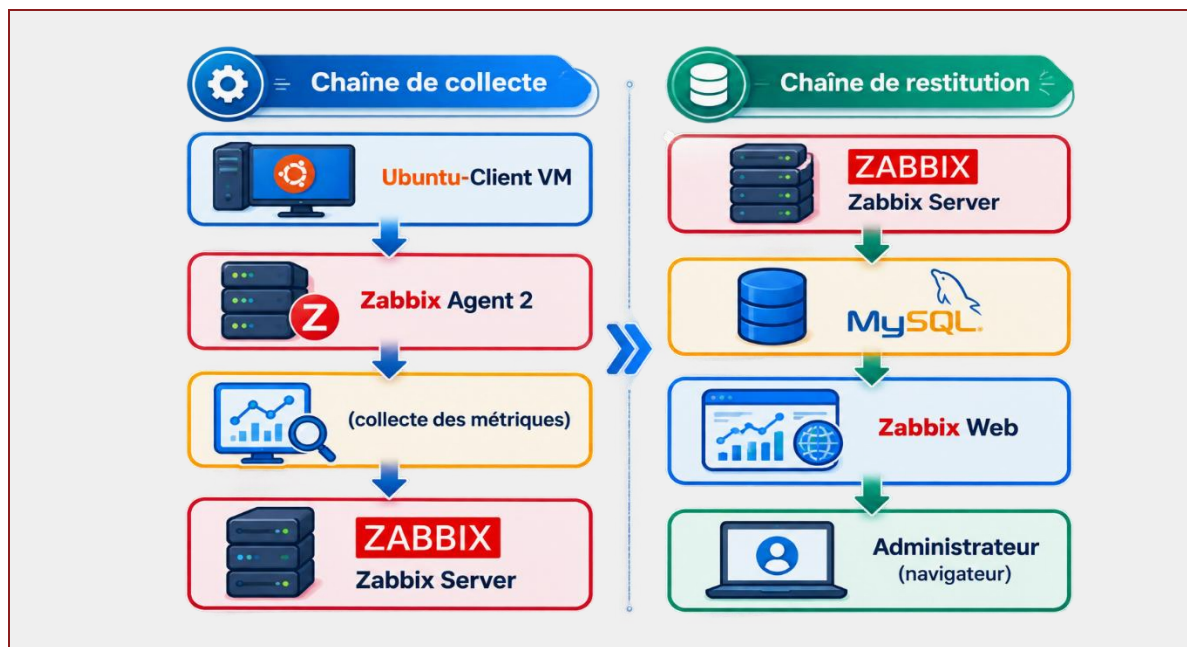


Figure 6 — Architecture logique de la solution

Ces deux chaînes convergent au niveau du Zabbix Server, point central de la solution : il reçoit les données, les confronte aux règles d'alerte, les persiste dans MySQL et pilote l'envoi des notifications.

3.3. Architecture physique du laboratoire

Sur le plan physique, le laboratoire est constitué de deux machines virtuelles distinctes, hébergées sur une même machine hôte via VMware Workstation. Zabbix-Server VM héberge la pile applicative conteneurisée ; Ubuntu-Client VM, l'hôte supervisé, héberge uniquement l'agent. Cette séparation reproduit fidèlement la logique d'un environnement de supervision réel.

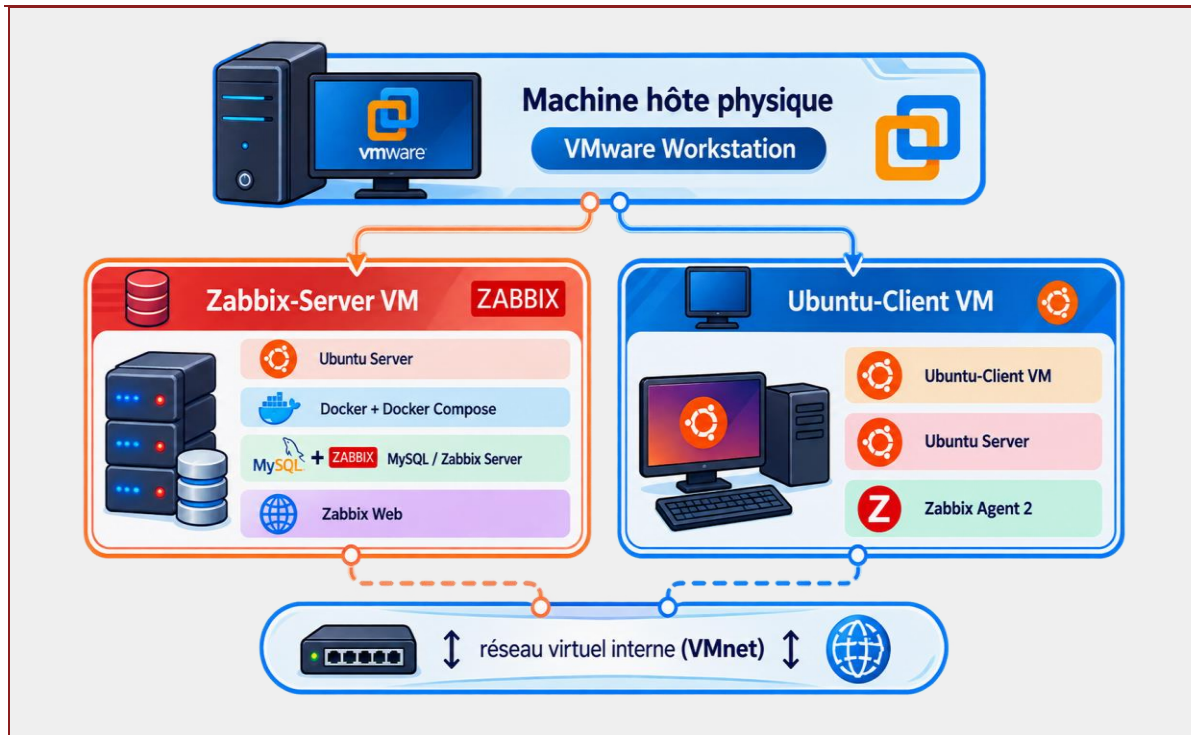


Figure 7 — Architecture physique du laboratoire

Cet environnement constitue un laboratoire de démonstration et non l'infrastructure de production de MedTech Group ; il a été conçu spécifiquement dans le cadre du stage pour l'apprentissage des concepts de supervision centralisée.

3.4. Architecture réseau

Les deux machines virtuelles sont reliées par un réseau virtuel interne configuré sous VMware, leur permettant de communiquer comme deux serveurs d'un même réseau local. Ubuntu-Client VM échange avec le Zabbix Server sur le port dédié à l'agent, tandis que Zabbix Web est accessible via le port HTTP exposé par le conteneur correspondant.

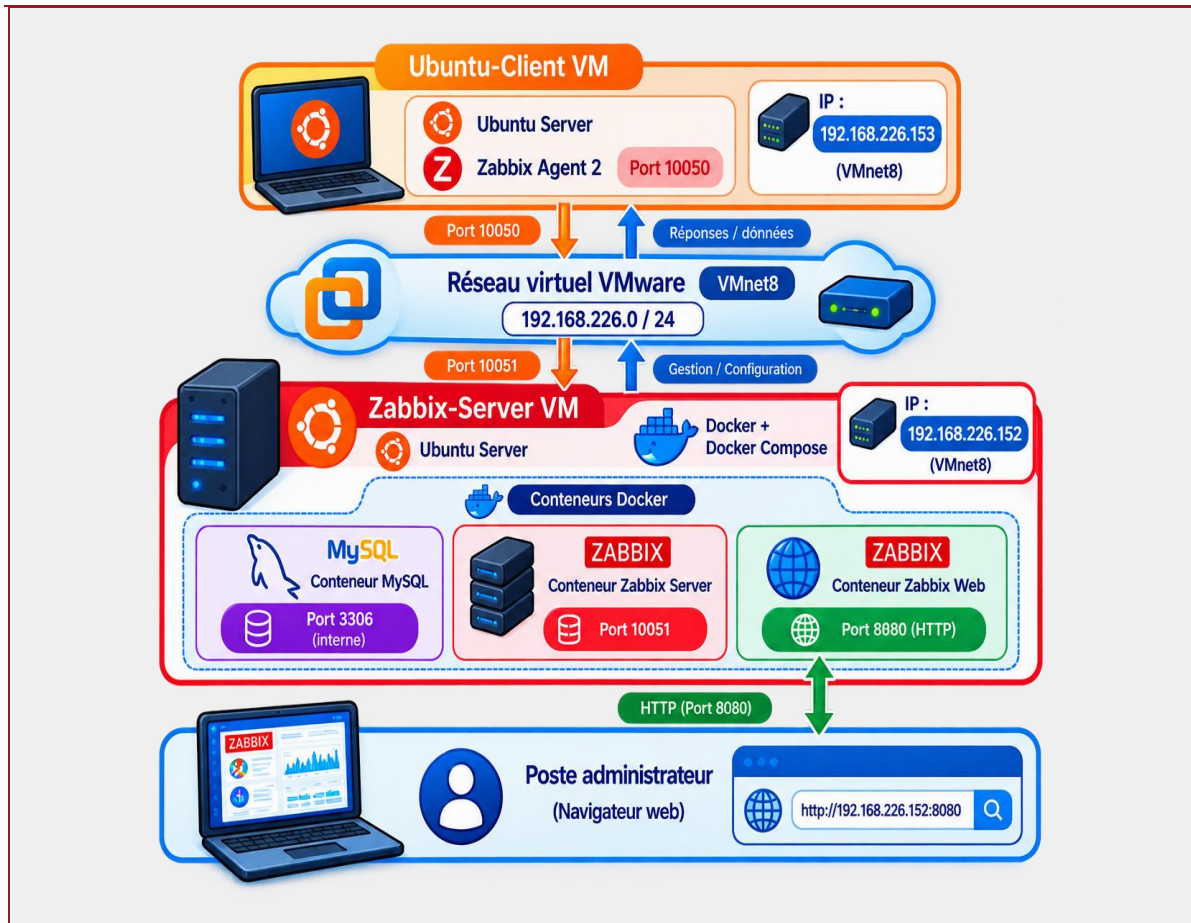


Figure 8 — Architecture réseau du laboratoire

Cette organisation distingue le port de collecte de l'agent (10050), le port de communication du serveur (10051) et le port HTTP de l'interface web. Le port 3306 de MySQL reste interne à l'environnement Docker et n'est pas exposé, limitant la surface d'exposition de la base de données.

3.5. Composants de la solution

Le tableau suivant synthétise l'ensemble des composants mobilisés dans la solution, ainsi que le rôle fonctionnel de chacun d'entre eux.

Tableau 1 — Composants de la solution et rôles associés

Composant	Nature	Rôle dans la solution
VMware Workstation	Hyperviseur	Hébergement et isolation des machines virtuelles du laboratoire
Ubuntu Server	Système d'exploitation	Socle des deux machines virtuelles (serveur et client)
Docker	Moteur de conteneurisation	Isolation et exécution des composants applicatifs Zabbix
Docker Compose	Orchestrateur	Description et déploiement coordonné de la pile Zabbix
MySQL	Système de gestion de base de données	Persistance des configurations, métriques et historiques Zabbix
Zabbix Server	Moteur de supervision	Collecte, traitement des règles d'alerte et pilotage des notifications
Zabbix Web	Interface d'administration	Configuration, visualisation et tableau de bord
Zabbix Agent 2	Agent de collecte	Remontée des métriques depuis l'hôte supervisé
Gmail (SMTP)	Service de messagerie	Canal de notification par courrier électronique
Microsoft Teams / Power Automate	Plateforme collaborative et automatisation	Canal de notification par webhook

3.6. Rôle du Zabbix Server

Le Zabbix Server constitue le composant central de la solution : il collecte et traite les données remontées par les agents, calcule les règles d'alerte (triggers) et déclenche les notifications. Il ne stocke pas directement les données mais s'appuie sur MySQL pour leur persistance, tout en pilotant le cycle de vie complet d'une alerte.

3.7. Rôle de Zabbix Agent 2

Zabbix Agent 2, évolution de l'agent historique réécrite pour de meilleures performances, collecte localement les indicateurs techniques et les transmet au serveur, en mode passif (le serveur interroge l'agent) ou actif (l'agent transmet spontanément selon un intervalle configuré).

3.8. Rôle de MySQL

MySQL assure la persistance de l'ensemble des données Zabbix : hôtes, modèles, historique des métriques, événements et configuration des notifications. Le choix d'une base

relationnelle correspond à la structuration intrinsèquement relationnelle des données de supervision.

3.9. Rôle de Zabbix Web

Zabbix Web constitue l'interface d'administration accessible via navigateur, permettant de configurer les objets de supervision, de consulter le tableau de bord et de visualiser l'historique des métriques. C'est depuis cette interface qu'ont été réalisées les configurations décrites au chapitre suivant.

3.10. Docker

Docker répond à un besoin d'isolation et de reproductibilité : chaque composant de la pile Zabbix s'exécute dans son propre conteneur, sans interférer avec les autres ni avec le système hôte, facilitant la maintenance puisque chaque conteneur peut être arrêté ou mis à jour indépendamment.

3.11. Docker Compose

Docker Compose décrit, dans un fichier unique, la configuration complète de la pile : images, variables d'environnement, volumes, réseau interne et ordre de démarrage garantissant que MySQL soit disponible avant le Zabbix Server. Cette description déclarative garantit la reconstruction identique de l'environnement.

3.12. Collecte des données

La collecte des données repose sur les items Zabbix, selon deux modalités : la collecte passive, où le serveur interroge l'agent, et la collecte active, où l'agent transmet spontanément ses données, permettant d'adapter le mode aux contraintes réseau et de charge.

3.13. Items

Un item représente un indicateur technique élémentaire collecté sur un hôte — charge processeur, espace disque, disponibilité d'un service. Chaque item est associé à une clé de collecte, une méthode et un intervalle de rafraîchissement, constituant la matière première des mécanismes de visualisation et d'alerte.

3.14. Triggers

Un trigger définit une expression logique sur un ou plusieurs items, dont l'évaluation détermine le passage d'un hôte à l'état de problème — par exemple un dépassement de seuil processeur ou l'indisponibilité d'un service. C'est le mécanisme central transformant une observation en événement à traiter.

3.15. Problems

Lorsqu'un trigger s'active, Zabbix génère un problem : un événement de supervision associé à une sévérité, un hôte et un horodatage, consultable dans Zabbix Web et point de départ des actions de notification.

3.16. Actions

Une action définit la réponse automatique à un problem : conditions de déclenchement, notification à un ou plusieurs utilisateurs, et éventuellement opérations à la résolution (recovery). C'est au niveau des actions que se fait le lien entre détection et notification effective.

3.17. Notifications

La notification transforme un événement technique en message reçu par un utilisateur sur un canal qu'il consulte au quotidien. Zabbix distingue le Media Type, canal générique (email, webhook, SMS), du User Media, qui associe un utilisateur à un Media Type et à des paramètres spécifiques. Deux canaux ont été mis en place : Gmail et Microsoft Teams.

3.18. Intégration Gmail

La notification par courrier électronique s'appuie sur un Media Type Email configuré avec les paramètres SMTP de Gmail, associé via un User Media au compte administrateur. Le schéma suivant résume la chaîne complète.

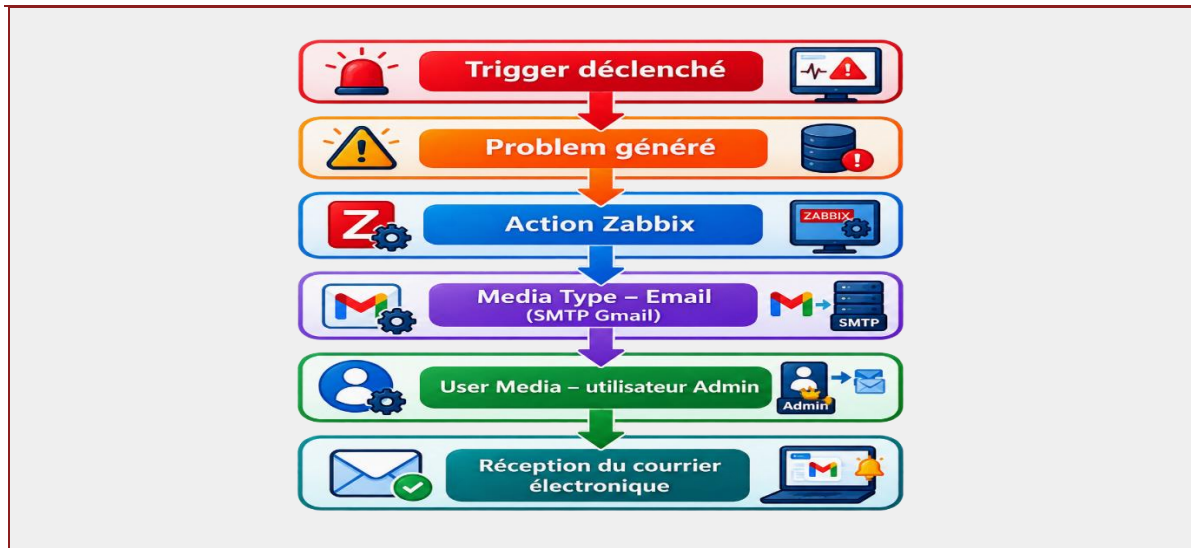


Figure 9 — Flux de notification par Gmail

3.19. Intégration Microsoft Teams Workflow

La notification vers Microsoft Teams repose sur un webhook entrant configuré via Power Automate. Un Workflow expose une URL capable de recevoir une requête HTTP et de la publier dans un canal Teams. Côté Zabbix, un Media Type Webhook transmet les données du problem détecté vers cette URL, associé de la même manière à l'administrateur via un User Media puis à une action.

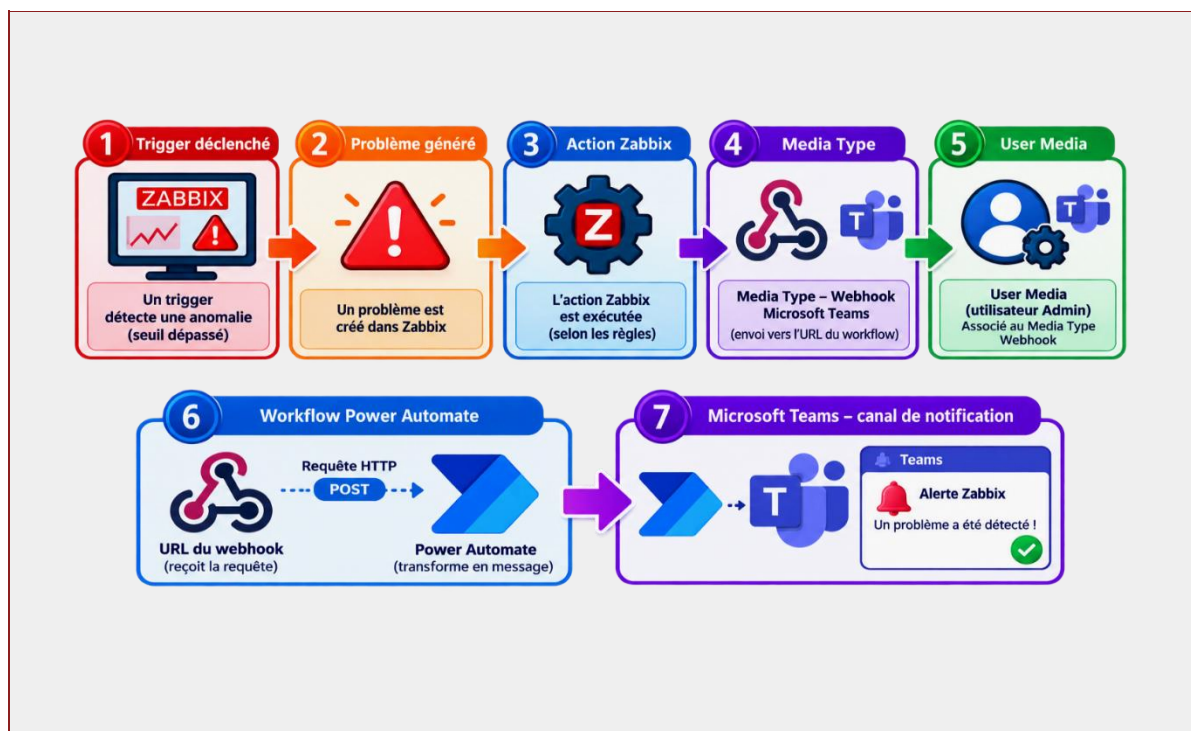


Figure 10 — Flux de notification par Microsoft Teams

Cette intégration illustre la flexibilité du Media Type Zabbix, qui s'interface via le mode Webhook avec pratiquement tout outil tiers disposant d'une API, répondant directement au besoin de notification multicanal du cahier des charges.

3.20. Sécurité

Bien que ce laboratoire soit un environnement de démonstration isolé, plusieurs principes de sécurité de base ont été respectés : MySQL n'est accessible que depuis le réseau interne des conteneurs ; les identifiants sont gérés via des variables d'environnement plutôt qu'en clair ; l'accès à Zabbix Web est protégé par authentification nominative. Ces principes devront être complétés, pour un déploiement en production, par les mesures décrites au chapitre 6, notamment HTTPS et TLS.

3.21. Flux de communication global

Le schéma suivant synthétise la chaîne complète de traitement d'une alerte, depuis la collecte de la métrique jusqu'à sa notification sur l'un des deux canaux configurés.

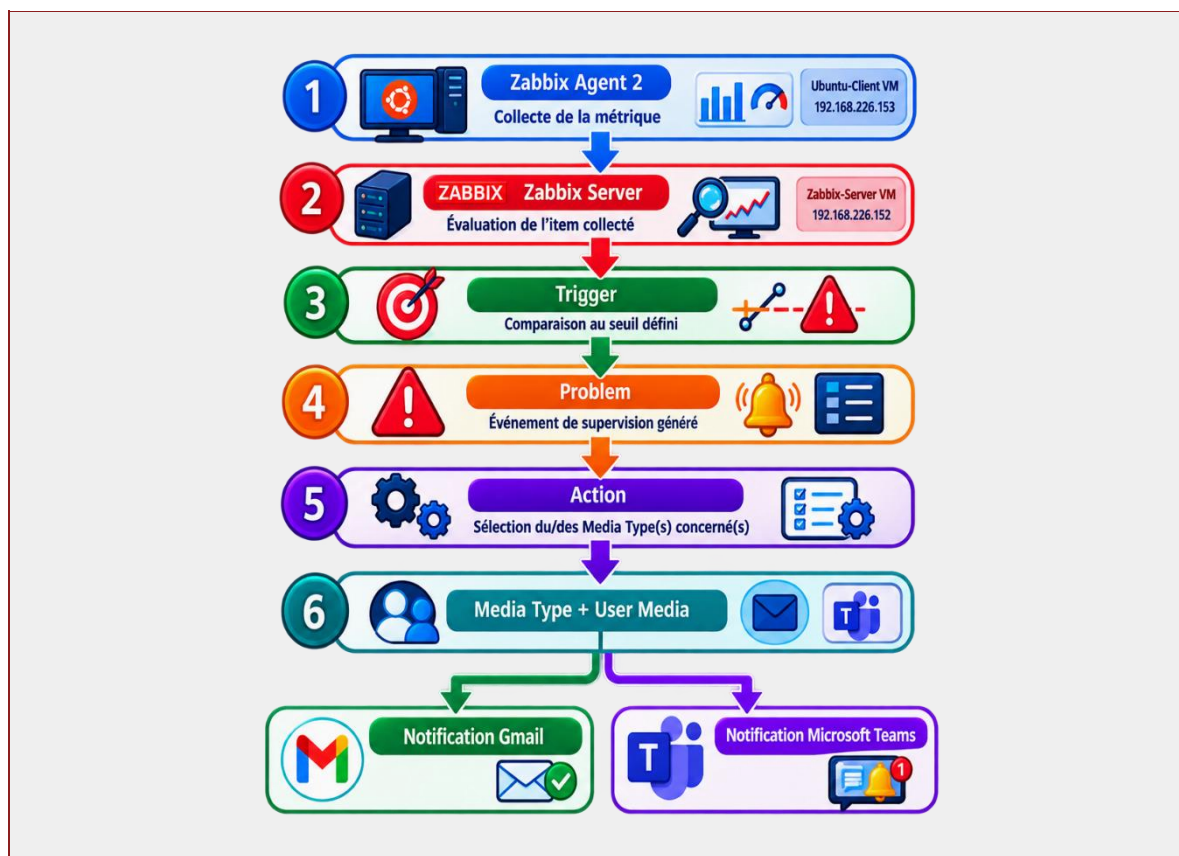


Figure 11 — Chaîne de traitement d'une alerte Zabbix

Conclusion du chapitre

Ce chapitre a détaillé la conception de la solution retenue : architectures générale, logique, physique et réseau, rôle de chaque composant, et deux chaînes de notification. Cette conception constitue la référence directe de la phase de mise en œuvre décrite au chapitre suivant.

Chapitre 4 — Mise en œuvre

Ce chapitre relate chronologiquement la mise en œuvre du laboratoire conçu au chapitre précédent : ce qui a été fait, pourquoi, comment, et avec quel résultat.

4.1. Préparation de l'environnement de virtualisation

La première étape a consisté à préparer l'environnement de virtualisation, afin de disposer de deux machines isolées reproduisant la séparation entre serveur de supervision et hôte supervisé, sans matériel physique dédié.

VMware Workstation a été installé sur la machine hôte, puis deux machines virtuelles ont été créées avec des ressources suffisantes pour exécuter respectivement la pile Zabbix conteneurisée et l'agent de supervision. Ubuntu Server a été installé en configuration minimale sur chacune, limitant la surface applicative.

Une fois les deux VMs opérationnelles, leur configuration réseau a été vérifiée : chaque machine a reçu une adresse IP au sein du réseau virtuel interne VMware. La connectivité a été validée par ping, confirmant l'absence de perte de paquets.

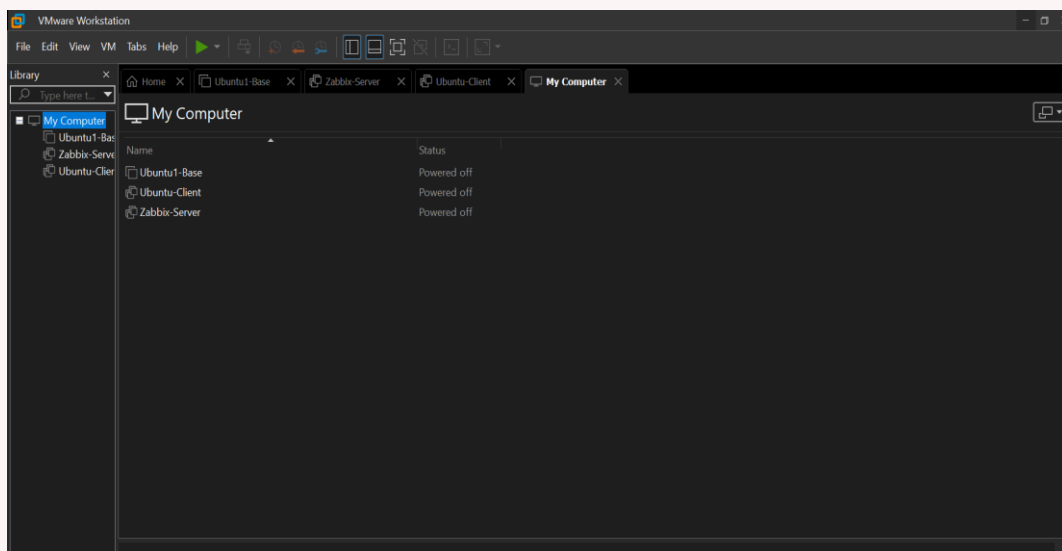


Figure 12 — Vérification de l'environnement VMware et des machines virtuelles (VMware Workstation, liste des VMs)

```
wys@ubuntu-client:~$ ip -br a
lo                UNKNOWN      127.0.0.1/8 ::1/128
ens33             UP            192.168.226.153/24 metric 100 fe80::20c:29ff:fe16:9503/64
wys@ubuntu-client:~$ ping -c 4 192.168.226.152
PING 192.168.226.152 (192.168.226.152) 56(84) bytes of data.
64 bytes from 192.168.226.152: icmp_seq=1 ttl=64 time=0.945 ms
64 bytes from 192.168.226.152: icmp_seq=2 ttl=64 time=1.82 ms
64 bytes from 192.168.226.152: icmp_seq=3 ttl=64 time=1.90 ms
64 bytes from 192.168.226.152: icmp_seq=4 ttl=64 time=2.02 ms

--- 192.168.226.152 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 0.945/1.670/2.023/0.425 ms
wys@ubuntu-client:~$

wys@zabbix-server:~$ ip -br a
lo                UNKNOWN      127.0.0.1/8 ::1/128
ens33             UP            192.168.226.152/24 metric 100 fe80::20c:29ff:fe42:f061/64
br-b63da5c2c893   UP            172.18.0.1/16 fe80::648c:adff:fe63:c1b5/64
docker0           DOWN          172.17.0.1/16
veth428f37f@if2   UP            fe80::5440:79ff:fe71:eb8a/64
veth66b983d@if2   UP            fe80::aca5:d5ff:fe9b:5c71/64
veth2107006@if2   UP            fe80::543e:6cff:fe1e:f412/64
wys@zabbix-server:~$ ping -c 4 192.168.226.153
PING 192.168.226.153 (192.168.226.153) 56(84) bytes of data.
64 bytes from 192.168.226.153: icmp_seq=1 ttl=64 time=1.08 ms
64 bytes from 192.168.226.153: icmp_seq=2 ttl=64 time=1.41 ms
64 bytes from 192.168.226.153: icmp_seq=3 ttl=64 time=1.71 ms
64 bytes from 192.168.226.153: icmp_seq=4 ttl=64 time=2.59 ms

--- 192.168.226.153 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 1.080/1.697/2.588/0.560 ms
wys@zabbix-server:~$
```

Figure 13 — Configuration réseau et test de connectivité (ping) entre Zabbix-Server VM et Ubuntu-Client VM

Résultat obtenu : les deux machines virtuelles sont opérationnelles, correctement adressées et communiquent sans erreur, constituant le socle du déploiement.

4.2. Installation et validation de Docker

La deuxième étape a consisté à installer Docker sur Zabbix-Server VM, l'ensemble des composants applicatifs devant être exécutés sous forme de conteneurs.

Docker Engine a été installé selon la procédure officielle, puis vérifié via la commande de version et l'exécution d'un conteneur de test. Docker Compose, packagé comme plugin, a également été installé et vérifié.

```
wys@zabbix-server:~$ docker --version
Docker version 29.6.0, build fb59821
wys@zabbix-server:~$ docker compose version
Docker Compose version v5.2.0
wys@zabbix-server:~$ sudo docker run hello-world
[sudo] password for wys:

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Figure 14 — Vérification de l'installation de Docker (docker --version, test de conteneur)

Résultat obtenu : Docker et Docker Compose sont fonctionnels sur Zabbix-Server VM, permettant de déployer la pile applicative.

4.3. Déploiement de la pile Zabbix via Docker Compose

Cette étape constitue le cœur du déploiement : un fichier Docker Compose a été rédigé décrivant MySQL, Zabbix Server et Zabbix Web, les volumes de persistance, le réseau interne et les variables d'environnement nécessaires.

Une attention particulière a été portée à l'ordre de démarrage, MySQL devant être initialisé avant le Zabbix Server, géré via les mécanismes de dépendance de Docker Compose. Le déploiement a été lancé en une seule commande.

L'état des conteneurs a été vérifié, confirmant que les trois services étaient démarrés et sains. Zabbix Web a ensuite été accédée depuis un navigateur, validant le bon fonctionnement de la chaîne applicative.

```
wys@zabbix-server:~$ cd /home/wys/zabbix-docker
wys@zabbix-server:~/zabbix-docker$ sudo docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
mysql-server	mysql:8.0	"docker-entrypoint.s..."	mysql-server	2 months ago	Up 19 minutes
zabbix-server	zabbix/zabbix-server-mysql:alpine-7.4-latest	"/usr/bin/docker-ent..."	zabbix-server	2 months ago	Up 19 minutes
zabbix-web	zabbix/zabbix-web-nginx-mysql:alpine-7.4-latest	"docker-entrypoint.sh"	zabbix-web	2 months ago	Up 19 minutes (healthy)

Figure 15 — Conteneurs actifs de la pile Zabbix

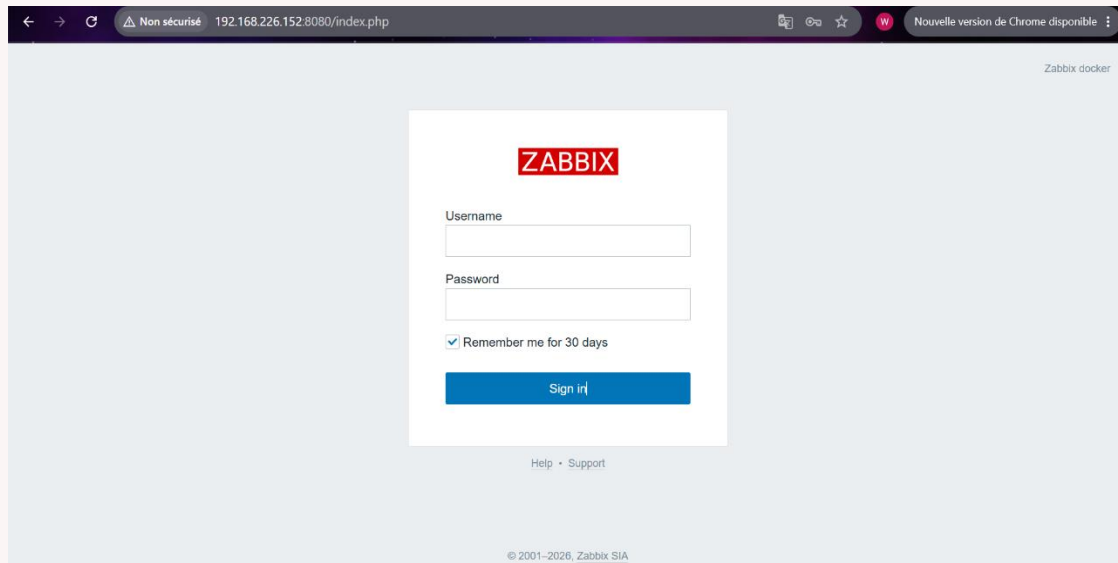


Figure 16 — Interface web Zabbix — page d'authentification

Résultat obtenu : la pile Zabbix est déployée et opérationnelle, l'interface d'administration accessible, marquant la mise en service du serveur de supervision.

4.4. Intégration de la machine cliente supervisée

4.4.1. Installation et configuration de l'agent Zabbix

Cette étape visait à intégrer Ubuntu-Client VM comme hôte supervisé. Zabbix Agent 2 y a été installé, puis configuré avec l'adresse du Zabbix Server et le nom d'hôte attendu dans Zabbix Web. Le service a été démarré et activé au démarrage.

4.4.2. Ajout de l'hôte et association du modèle

Un nouvel hôte a été créé dans Zabbix Web, correspondant à Ubuntu-Client VM, avec son adresse IP renseignée comme interface de l'agent. Un modèle standard, regroupant les indicateurs Linux courants, lui a été associé.

4.4.3. Difficulté rencontrée : indisponibilité de l'agent Zabbix

Une fois l'hôte créé, Zabbix Web a signalé une indisponibilité affichant « Linux: Zabbix agent is not available » pour Ubuntu-Client VM, indiquant que le serveur n'était pas parvenu à communiquer avec l'agent, malgré une configuration apparemment correcte.

☐	Time	Severity	Recovery time	Status	Info	Host	Problem	Duration	Update	Actions
☐	03:15:05 PM	Average		PROBLEM		Ubuntu-Client VM	Linux: Zabbix agent is not available (for 3m)	1m 5s	Update	2

Figure 17 — Message d'indisponibilité de l'agent Zabbix sur l'hôte Ubuntu-Client VM

Une démarche de diagnostic a été engagée, vérifiant successivement l'état du service agent, l'exactitude de l'adresse du serveur et du nom d'hôte déclarés, ainsi que l'accessibilité réseau du port utilisé par l'agent.

Cette analyse a identifié les causes les plus probables : défaut de correspondance du nom d'hôte, restriction du pare-feu local, ou service non redémarré après modification. La correction de ces éléments — redémarrage du service, alignement du nom d'hôte, vérification du pare-feu — a rétabli la communication, confirmée par le passage de l'hôte à l'état disponible.

Name	Interface	Availability	Tags	Status	Latest data	Problems	Graphs	Das
Ubuntu-Client VM	192.168.226.153:10050	ZBX	class: os target: linux	Enabled	Latest data 68	Problems	Graphs 14	Das
Zabbix-Server VM	192.168.226.152:10050	ZBX	class: os class: software subclass: logging	Enabled	Latest data 146	Problems	Graphs 14	Das

Figure 18 — Hôte Ubuntu-Client VM après résolution de l'incident (agent disponible)

Cette difficulté, mineure au regard du projet, illustre une situation représentative : une configuration individuellement correcte peut, une fois assemblée, produire un dysfonctionnement nécessitant un diagnostic méthodique plutôt qu'une simple relecture de documentation.

Dashboard Host IPMI Tags Macros Inventory Encryption Value mapping

* Host name

Visible name

Templates Name Actions

[Linux by Zabbix agent](#) [Unlink](#) [Unlink and clear](#)

* Host groups [Linux servers](#)

Interfaces	Type	IP address	DNS name	Connect to	Port
Agent		192.168.226.153		IP DNS	10050

[Add](#)

Description

Name ▲ Items Triggers Graphs Discovery Web Interface Proxy Templates Status Available

Ubuntu-Client VM Items 68 Triggers 25 Graphs 14 Discovery 3 Web 192.168.226.153:10050 Linux by Zabbix agent Enabled ZBX

Figure 19 — Fiche de l'hôte Ubuntu-Client dans Zabbix (configuration de l'interface agent)

Template

Template Tags 2 Macros 27 Value mapping 4

* Template name

Visible name

Templates

* Template groups [Templates/Operating systems](#)

Description

Notes on filesystem (FS) discovery:

- The ext4/3/2 filesystem reserves space for privileged usage, typically set at 5% by default.
- BTRFS allocates a default of 10% of the volume for its own needs.
- To mitigate potential disasters, FS usage triggers are based on the maximum

Vendor and version Zabbix, 7.4-3

Figure 20 — Modèle (template) associé à l'hôte supervisé

4.5. Configuration du tableau de bord et des alertes

Une fois la collecte validée, un tableau de bord a été configuré regroupant les indicateurs de l'hôte. Des triggers ont été définis, notamment sur la disponibilité de l'agent, générant automatiquement un problem en cas d'interruption.

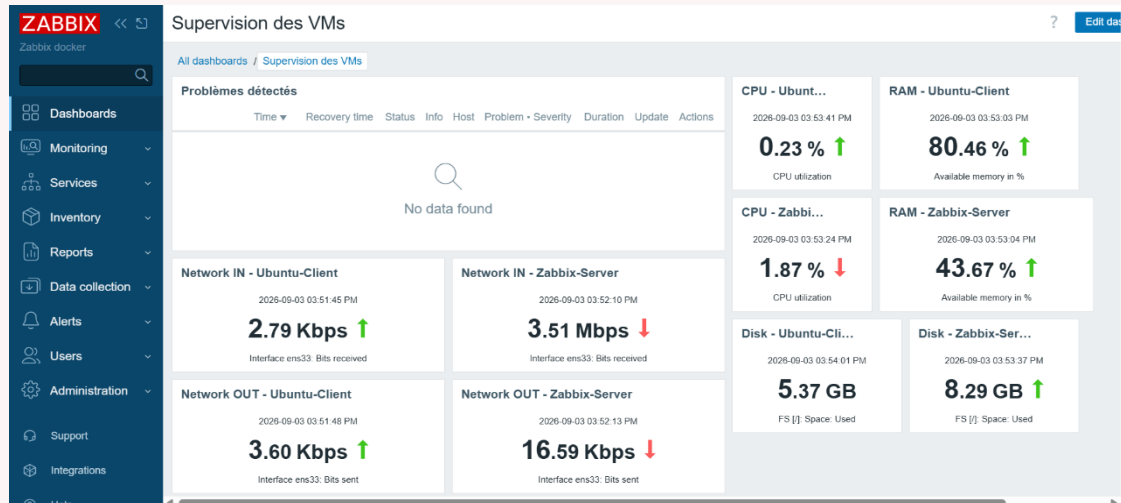


Figure 21 — Tableau de bord (Dashboard) Zabbix présentant les métriques de l'hôte supervisé

Résultat obtenu : le tableau de bord restitue en continu l'état de l'hôte, et les triggers détectent automatiquement toute anomalie.

4.6. Intégration des canaux de notification

4.6.1. Notification par Gmail

Un Media Type Email a été configuré avec les paramètres SMTP de Gmail, associé via un User Media au compte administrateur. Une action a été configurée pour déclencher un email à chaque nouveau problem.

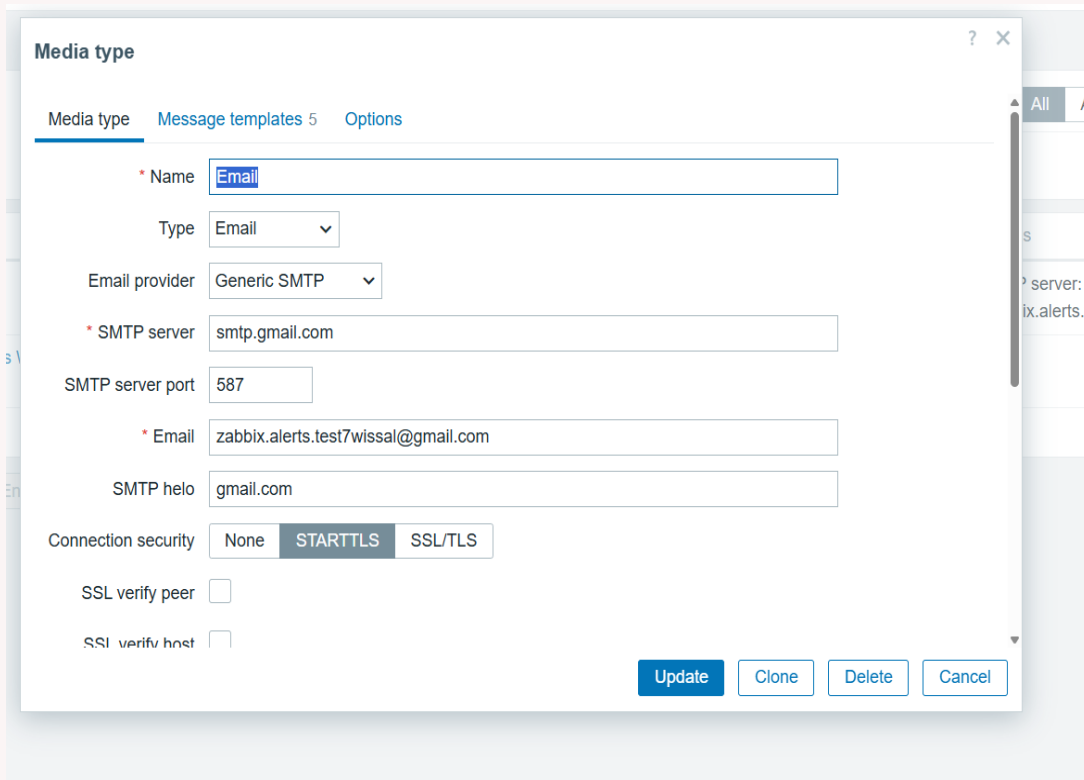


Figure 22 — Configuration du Media Type Email (paramètres SMTP Gmail)

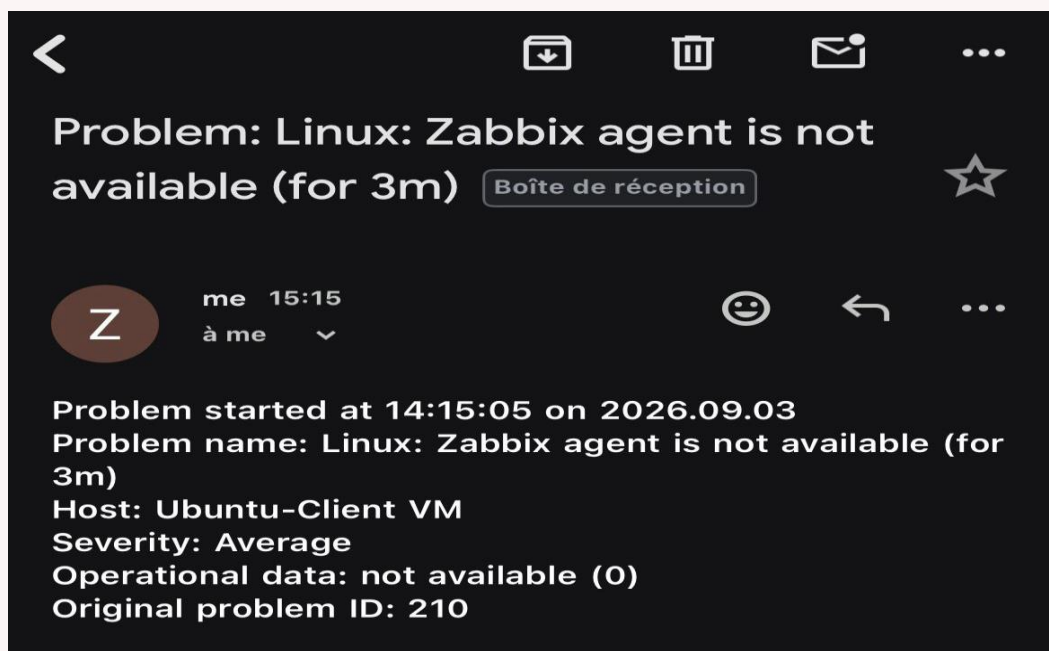
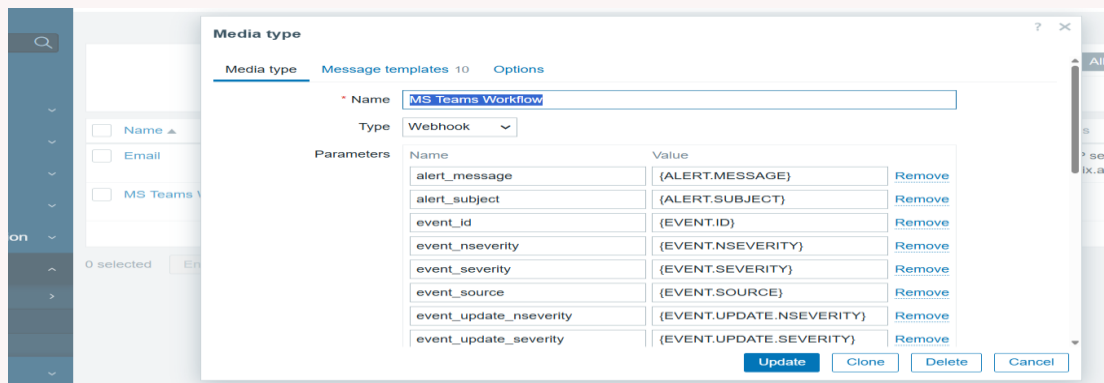


Figure 23 — Réception d'une notification par courrier électronique

4.6.2. Notification par Microsoft Teams

Un Workflow Power Automate a été créé, exposant une URL de webhook publiant un message dans un canal Teams. Un Media Type Webhook, côté Zabbix, transmet les informations du problème détecté vers cette URL, associé de la même manière à l'administrateur puis à une action.

☐ [MS Teams Workflow](#) Webhook **Enabled** 5 Notifications Teams - Alerts Zabbix, Report not supported items, Report not supported low level discovery rules, Report problems to Zabbix administrators, Report unknown triggers



The screenshot shows the 'Media type' configuration window in Zabbix. The 'Name' field is 'MS Teams Workflow' and the 'Type' is 'Webhook'. Below, a table lists parameters to be sent in the webhook message:

Name	Value	Action
alert_message	{ALERT.MESSAGE}	Remove
alert_subject	{ALERT.SUBJECT}	Remove
event_id	{EVENT.ID}	Remove
event_nseverity	{EVENT.NSEVERITY}	Remove
event_severity	{EVENT.SEVERITY}	Remove
event_source	{EVENT.SOURCE}	Remove
event_update_nseverity	{EVENT.UPDATE.NSEVERITY}	Remove
event_update_severity	{EVENT.UPDATE.SEVERITY}	Remove

Buttons at the bottom: Update, Clone, Delete, Cancel.

Figure 24 — Configuration du Media Type Webhook Microsoft Teams dans Zabbix

Le test de cette intégration a permis d'obtenir avec succès une notification sur Microsoft Teams, confirmant le bon fonctionnement de la chaîne webhook — résultat présenté à l'encadrant professionnel, qui l'a validé de manière informelle.

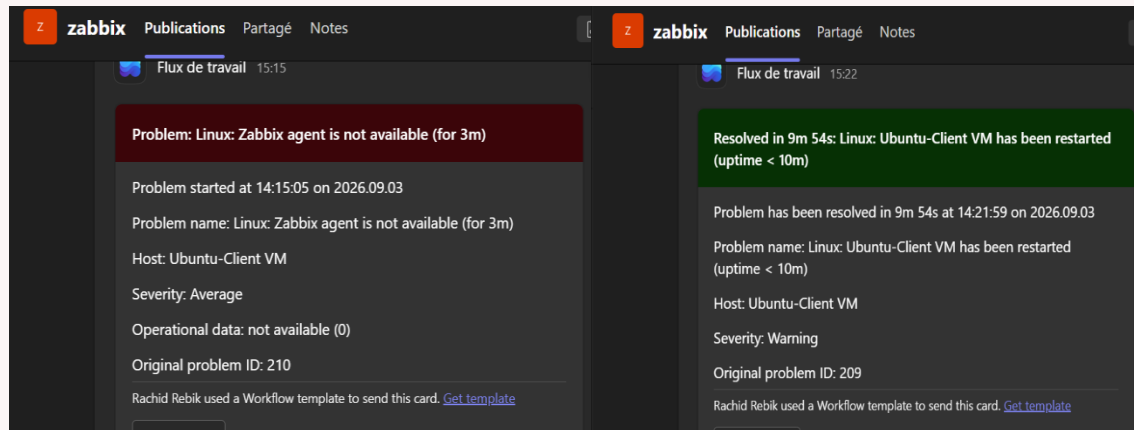


Figure 25 — Réception d'une notification sur Microsoft Teams

4.7. Synthèse des étapes de mise en œuvre

La mise en œuvre s'est déroulée selon une progression maîtrisée, chaque étape validée avant la suivante, permettant d'isoler et de résoudre méthodiquement les difficultés rencontrées, notamment celle de l'agent Zabbix.

Conclusion du chapitre

Ce chapitre a présenté chronologiquement le passage de la conception théorique à un laboratoire opérationnel. La difficulté rencontrée sur l'agent Zabbix illustre la dimension pratique et itérative de tout projet d'ingénierie.

Chapitre 5 — Tests, résultats et validation

Ce chapitre présente la stratégie de tests mise en œuvre, les résultats obtenus et leur analyse critique au regard des objectifs fixés en début de stage.

5.1. Stratégie de test

La stratégie de test repose sur une vérification progressive de chaque maillon, de la connectivité réseau jusqu'à la réception effective des notifications, permettant d'isoler rapidement tout composant en cause.

Les tests couvrent successivement la connectivité réseau, le fonctionnement de Docker, la disponibilité de l'agent, la configuration de l'hôte, la restitution dans le tableau de bord, le déclenchement des triggers, puis la réception des notifications.

5.2. Plan de tests

Le tableau suivant présente le plan de tests détaillé, précisant l'objectif, la méthode, le résultat attendu, le résultat obtenu et le statut de chaque test.

Tableau 2 — Plan de tests fonctionnels et techniques

ID	Test	Objectif	Méthode	Résultat attendu	Résultat obtenu	Statut
T01	Connectivité réseau	Vérifier la communication entre les deux VMs	Commande ping entre Zabbix-Server VM et Ubuntu-Client VM	Réponse sans perte de paquets	Réponse obtenue, aucune perte	Succès
T02	Fonctionnement Docker	Vérifier l'installation de Docker	Commande de version + conteneur de test	Docker opérationnel	Version affichée, conteneur exécuté	Succès
T03	État des conteneurs	Vérifier le démarrage de la pile Zabbix	docker compose ps	3 conteneurs actifs (MySQL, Server, Web)	3 conteneurs actifs et sains	Succès
T04	Accès interface web	Vérifier l'accessibilité de Zabbix Web	Connexion via navigateur	Page d'authentification accessible	Page affichée, connexion possible	Succès
T05	Disponibilité de l'agent	Vérifier la communication Serveur ↔ Agent	Observation du statut de l'hôte	Statut disponible	Indisponible initialement, corrigé	Succès (après correction)

ID	Test	Objectif	Méthode	Résultat attendu	Résultat obtenu	Statut
T06	Collecte des métriques	Vérifier la remontée des indicateurs	Consultation des dernières données de l'hôte	Métriques mises à jour périodiquement	Métriques collectées avec succès	Succès
T07	Affichage Dashboard	Vérifier la restitution visuelle	Consultation du tableau de bord	Indicateurs affichés correctement	Tableau de bord fonctionnel	Succès
T08	Déclenchement Trigger	Vérifier la génération d'un Problem	Simulation d'une condition de seuil	Apparition d'un Problem dans l'interface	Problem généré comme attendu	Succès
T09	Notification Gmail	Vérifier la réception d'un email	Déclenchement de l'action associée	Email reçu avec les détails du Problem	Email reçu avec succès	Succès
T10	Notification Microsoft Teams	Vérifier la réception via webhook	Déclenchement de l'action associée	Message publié dans le canal Teams	Notification Teams reçue avec succès	Succès

5.3. Validation des objectifs

Le tableau suivant met en regard les objectifs fixés au chapitre 2 avec leur niveau de réalisation et les preuves associées.

Tableau 3 — Validation des objectifs du stage

Objectif	Réalisation	Preuve
Comprendre les principes de la supervision informatique	Atteint	Étude comparative des solutions (chapitre 2) et conception argumentée (chapitre 3)
Concevoir une architecture de supervision cohérente	Atteint	Architectures générale, logique, physique et réseau (chapitre 3)
Déployer la pile Zabbix conteneurisée	Atteint	Conteneurs actifs (T03), interface accessible (T04)
Intégrer un hôte supervisé via Zabbix Agent 2	Atteint (après diagnostic)	Statut de l'hôte disponible (T05), métriques collectées (T06)
Configurer des alertes automatisées	Atteint	Déclenchement du trigger et génération du Problem (T08)
Notifier par courrier électronique	Atteint	Réception effective de l'email (T09)
Notifier via Microsoft Teams	Atteint	Réception effective de la notification Teams (T10)
Déployer la solution sur l'infrastructure de production	Non traité (hors périmètre du stage d'observation)	Présenté comme perspective d'évolution (chapitre 6)

5.4. Analyse des résultats

L'ensemble des tests confirme le bon fonctionnement de la chaîne de supervision : la collecte s'effectue correctement, les seuils se déclenchent comme attendu, et les deux canaux délivrent effectivement les alertes, validant la cohérence entre conception et mise en œuvre.

Le seul test ayant nécessité une correction concerne la disponibilité de l'agent (T05), résolu selon la démarche décrite en 4.4.3. Ce résultat illustre l'utilité du processus de test : c'est la vérification systématique qui a permis d'identifier ce dysfonctionnement avant qu'il ne compromette le projet.

5.5. Difficultés et analyse critique

Au-delà de cette difficulté, le projet a mis en évidence un point d'attention plus général : la configuration coordonnée de plusieurs composants distincts exige une rigueur particulière, une simple incohérence suffisant à interrompre la chaîne sans message d'erreur explicite.

Par ailleurs, le périmètre du laboratoire, volontairement limité à deux machines virtuelles, ne permet pas de valider le comportement de la solution à l'échelle d'une infrastructure de production comptant des dizaines d'hôtes — une limite assumée dans le cadre d'un stage d'observation.

5.6. Limites

Les principales limites peuvent être résumées ainsi : absence de chiffrement HTTPS/TLS, absence de sauvegarde automatisée de MySQL, absence de test de charge, et périmètre limité aux systèmes sans supervision des équipements réseau via SNMP.

Conclusion du chapitre

La phase de tests a validé l'ensemble de la chaîne de supervision, de la collecte des métriques à la notification effective des alertes, tout en objectivant la difficulté rencontrée et les limites du travail réalisé.

Chapitre 6 — Bilan et perspectives

Ce dernier chapitre dresse le bilan du stage — compétences acquises, difficultés rencontrées, apports — avant de présenter les perspectives d'évolution vers un déploiement en production.

6.1. Compétences techniques acquises

Ce stage a permis de développer une pratique concrète de la virtualisation avec VMware, ainsi que l'administration système Linux : gestion des services, configuration réseau et installation de paquets en ligne de commande.

La conteneurisation avec Docker et Docker Compose a constitué un axe central d'apprentissage, à travers la rédaction d'une configuration multi-services et la manipulation des commandes de diagnostic courantes. La supervision a été abordée en profondeur via Zabbix : hôtes, modèles, cycle de vie d'un item, d'un trigger et d'un problem, et mise en place de deux mécanismes de notification distincts.

6.2. Compétences professionnelles

Au-delà du technique, ce stage a permis de développer une méthodologie structurée, fondée sur la progression validée par étapes et un diagnostic rigoureux, ainsi qu'une capacité à présenter des choix techniques et à intégrer des retours constructifs.

6.3. Difficultés rencontrées et solutions apportées

La principale difficulté concerne l'indisponibilité initiale de l'agent Zabbix, détaillée en 4.4.3. Le tableau suivant en propose une synthèse, ainsi que d'autres difficultés méthodologiques rencontrées.

Tableau 4 — Synthèse des difficultés rencontrées et solutions apportées

Difficulté rencontrée	Analyse	Solution apportée
Agent Zabbix indisponible sur Ubuntu-Client VM (« Zabbix agent is not available »)	Absence de communication entre le serveur et l'agent, malgré une configuration apparemment correcte de l'hôte et du modèle	Vérification et correction du nom d'hôte, du service agent et des règles de pare-feu locales ; redémarrage du service

Difficulté rencontrée	Analyse	Solution apportée
Coordination du démarrage des conteneurs (MySQL, Zabbix Server, Zabbix Web)	Risque de démarrage du serveur avant l'initialisation complète de la base de données	Utilisation des mécanismes de dépendance de Docker Compose
Configuration du Media Type Webhook pour Microsoft Teams	Nécessité d'adapter le format des données transmises au format attendu par le Workflow Power Automate	Analyse de la documentation officielle Zabbix relative aux Media Types Webhook et ajustement de la configuration

6.4. Apports du stage

Ce stage a permis de passer d'une connaissance théorique de la supervision à une pratique concrète, de la conception aux tests. Il a aussi permis de mieux comprendre le fonctionnement d'un groupe tel que MedTech Group et les enjeux du périmètre Infrastructure, Systèmes et DevOps.

6.5. Limites du travail réalisé

Le travail réalisé reste circonscrit à un laboratoire à échelle réduite, ne couvrant qu'un hôte et deux canaux de notification, sans sécurisation avancée ni haute disponibilité — des limites cohérentes avec le cadre d'un stage d'observation.

6.6. Perspectives d'évolution

Plusieurs perspectives peuvent rapprocher la solution d'un déploiement en production. Les éléments suivants constituent des pistes envisagées, non des réalisations effectuées durant le stage.

- Sécurisation des communications par la mise en place du protocole HTTPS et de TLS sur l'interface Zabbix Web ainsi que sur les échanges entre composants.
- Extension du périmètre de supervision aux équipements réseau (switchs, routeurs, pare-feux) via le protocole SNMP, complémentaire à la supervision par agent.
- Mise en place d'un ou plusieurs Zabbix Proxy, permettant de répartir la charge de collecte et de superviser des sites distants sans connexion directe entre chaque hôte et le serveur central.
- Mise en œuvre d'une stratégie de sauvegarde régulière de la base de données MySQL, condition indispensable à la continuité de la solution en environnement de production.

-
- Étude d'une architecture en haute disponibilité pour le Zabbix Server et la base de données, afin de garantir la continuité de la supervision elle-même.
 - Développement de modèles (templates) personnalisés, adaptés aux applications métiers spécifiques de MedTech Group, au-delà des modèles standards utilisés dans le laboratoire.
 - Affinement de la gestion des seuils d'alerte, notamment par l'introduction de seuils dynamiques ou de fenêtres de tolérance, afin de réduire le risque de fausses alertes.
 - Extension de la supervision aux environnements Cloud et à des infrastructures réparties sur plusieurs sites géographiques.
 - Déploiement progressif et encadré de la solution sur un périmètre restreint de l'infrastructure réelle de MedTech Group, en conditions contrôlées, préalablement à toute généralisation.

Conclusion du chapitre

Ce chapitre a dressé un bilan structuré du stage — compétences, difficultés surmontées, limites assumées — et tracé un chemin cohérent entre le laboratoire réalisé et un éventuel déploiement futur en production.

Conclusion générale

Ce rapport a présenté la conception d'une solution centralisée de supervision, réalisée dans le cadre d'un stage d'observation au sein de MedTech Group. La problématique de supervision proactive a trouvé une réponse concrète à travers un laboratoire fondé sur Zabbix.

La démarche a couvert l'ensemble des étapes d'un travail d'ingénierie : étude préalable justifiant le choix de Zabbix, conception des architectures, mise en œuvre chronologique incluant la résolution méthodique d'une difficulté technique, et tests validant la chaîne complète jusqu'à la réception des alertes.

Les résultats démontrent la faisabilité de l'architecture conçue : le laboratoire, bien que circonscrit à deux machines virtuelles, reproduit fidèlement les principes d'une solution de supervision centralisée applicable à plus grande échelle.

Ce stage a permis de développer des compétences concrètes en virtualisation, administration Linux, conteneurisation et supervision, ainsi que des compétences professionnelles de rigueur méthodologique. Il a mis en évidence les limites propres au laboratoire et la nécessité, pour la production, de traiter la sécurisation, la haute disponibilité et l'extension aux équipements réseau.

Ce travail constitue une première étape cohérente vers la mise en place d'une supervision centralisée, et ouvre des perspectives claires au-delà du cadre de ce stage d'observation.

Bibliographie

Articles scientifiques

- [1] Bhanage D.A., Pawar A.V., Kotecha K. « *IT Infrastructure Anomaly Detection and Failure Handling: A Systematic Literature Review Focusing on Datasets, Log Preprocessing, Machine & Deep Learning Approaches and Automated Tools* ». IEEE Access, 2021;9:156392-156421. DOI: 10.1109/ACCESS.2021.3128283
- [2] Bernstein D. « *Containers and Cloud: From LXC to Docker to Kubernetes* ». IEEE Cloud Computing, 2014;1(3):81-84. DOI: 10.1109/MCC.2014.51
- [3] Pahl C., Brogi A., Soldani J., Jamshidi P. « *Cloud Container Technologies: A State-of-the-Art Review* ». IEEE Transactions on Cloud Computing, 2019;7(3):677-692. DOI: 10.1109/TCC.2017.2702586
- [4] Felter W., Ferreira A., Rajamony R., Rubio J. « *An Updated Performance Comparison of Virtual Machines and Linux Containers* ». 2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), p. 171-172. DOI: 10.1109/ISPASS.2015.7095802
- [5] Boettiger C. « *An Introduction to Docker for Reproducible Research* ». ACM SIGOPS Operating Systems Review, 2015;49(1):71-79. DOI: 10.1145/2723872.2723882
- [6] Merkel D. « *Docker: Lightweight Linux Containers for Consistent Development and Deployment* ». Linux Journal, 2014;2014(239), article 2.

Documentation technique

- [7] Zabbix SIA. « *Zabbix Documentation* ». Documentation technique officielle, versions courantes.
- [8] Docker Inc. « *Docker Documentation* ». Documentation technique officielle relative à Docker Engine et Docker Compose.
- [9] Microsoft Corporation. « *Power Automate Documentation — Workflows et connecteurs Microsoft Teams* ». Documentation technique officielle.
- [10] Canonical Ltd. « *Ubuntu Server Documentation* ». Documentation technique officielle.
- [11] Broadcom / VMware. « *VMware Workstation — Guide de l'utilisateur* ». Documentation technique officielle.
- [12] MedTech Group. « *Présentation institutionnelle du Groupe MedTech : historique, activités et filiales* ». Document institutionnel.

Annexes

Les annexes suivantes regroupent des éléments techniques complémentaires : commandes Linux et Docker utilisées durant le stage, extrait du fichier Docker Compose ayant permis le déploiement de la pile Zabbix, ainsi qu'un tableau récapitulatif de tests complémentaires.

Annexe A — Commandes Linux utiles

```
# Vérification de la connectivité réseau entre les deux VMs
ping -c 4 <adresse_ip_destination>

# Vérification de l'état d'un service (exemple : agent Zabbix)
sudo systemctl status zabbix-agent2
sudo systemctl restart zabbix-agent2
sudo systemctl enable zabbix-agent2

# Consultation des journaux du service agent
sudo journalctl -u zabbix-agent2 -f

# Vérification des ports en écoute
sudo ss -tulnp | grep 10050
```

Annexe B — Commandes Docker

```
# Vérification de la version de Docker
docker --version
docker compose version

# Démarrage de la pile applicative Zabbix
docker compose up -d

# Vérification de l'état des conteneurs
docker ps
docker compose ps

# Consultation des journaux d'un conteneur
docker logs -f zabbix-server

# Arrêt de la pile applicative
docker compose down
```

Annexe C — Extrait du fichier Docker Compose

L'extrait suivant illustre la structure générale du fichier docker-compose.yml utilisé pour le déploiement de la pile Zabbix (services principaux, sans les valeurs sensibles).

```
services:
  mysql-server:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: zabbix
      MYSQL_USER: zabbix
      MYSQL_PASSWORD: "*****"
    volumes:
      - mysql-data:/var/lib/mysql

  zabbix-server:
    image: zabbix/zabbix-server-mysql:latest
    depends_on:
      - mysql-server
    environment:
      DB_SERVER_HOST: mysql-server
      MYSQL_USER: zabbix
      MYSQL_PASSWORD: "*****"
    ports:
      - "10051:10051"

  zabbix-web:
    image: zabbix/zabbix-web-nginx-mysql:latest
    depends_on:
      - zabbix-server
      - mysql-server
    ports:
      - "80:8080"

volumes:
  mysql-data:
```

Annexe D — Tableau complémentaire de tests

Tableau 5 — Tests complémentaires réalisés en environnement de laboratoire

ID	Test	Résultat obtenu
T11	Redémarrage du service agent après modification de configuration	Prise en compte effective de la nouvelle configuration
T12	Persistance des données après redémarrage des conteneurs	Données conservées grâce au volume Docker dédié à MySQL
T13	Cohérence du nom d'hôte entre l'agent et l'interface Zabbix Web	Correction appliquée, cohérence rétablie